

Lesson 1	4
P1	4
P2	5
P3	7
P4	8
P5	9
P6	10
Lesson 2	12
P1	12
P2	13
P3	14
P4	15
P5	17
P6	18
Lesson 3	19
P1	19
P2	20
P3	21
P4	22
P5	24
P6	25
Lesson 4	26
P1	26
P2	27
P3	28
P4	29
P5	30
P6	31
Lesson 5	32
P1	32
P2	33
P3	34
P4	35
P5	36
P6	37
Lesson 6	37
P1	38
P2	38

P3	39
P4	40
P5	41
P6	42
Lesson 7	43
P1	43
P2	44
P3	44
P4	45
P5	46
P6	47
Lesson 8	48
P1	48
P2	49
P3	49
P4	50
P5	51
P6	52
Lesson 9	53
P1	53
P2	54
P3	55
P4	56
P5	57
P6	58
Lesson 10	59
P1	59
P2	60
P3	61
P4	62
P5	63
P6	63
Lesson 11	64
P1	65
P2	65
P3	66
P4	67
P5	68
P6	69
Lesson 12	70
P1	70

P2	71
P3	72
P4	73
P5	74
P6	75
Lesson 13	76
P1	76
P2	77
P3	78
P4	79
P5	81
P6	82
Lesson 14	83
P1	83
P2	85
P3	86
P4	87
P5	89
P6	90
Lesson 15	92
P1	92
P2	93
P3	95
P4	96
P5	98
P6	99
Lesson 16	101
P1	101
P2	102
P3	104
P4	105
P5	107
P6	108
Lesson 17	110
P1	110
P2	111
P3	113
P4	114
P5	116
P6	117

Lesson 1

P1

Act as an expert frontend game developer and early-years UX designer. Write a complete game called "Eco-Sorter Detective" designed for 6-year-old students.

The game must be fully playable, tablet-friendly, and self-contained in ONE code block containing HTML, CSS (in `<style>`), and JavaScript (in `<script>`). Do not use external assets. Use system emojis as the primary graphics and a bright, cheerful, high-contrast palette. Clear instructions so users know how to play.

Strict Tablet & Interface Constraints:

1. Tap-to-Route Interface: Do NOT use dragging mechanics. Display the active waste item in a giant center "AI Scanner Box" (minimum 120px x 120px). Below or beside it, display the available Bins as massive, colored button targets (at least 100px x 100px). The student routes the item simply by tapping the correct bin button.
2. Unlocked Progression Matrix: Display a horizontal row of buttons at the very top: "Level 1" to "Level 5". Students must be able to tap and play any level instantly in any order.
3. No Negative Reinforcement: If a child makes a mistake, do NOT subtract points. Animate the item with a gentle shake, keep it in the scanner, and display a clear text speech bubble from Sorty the AI explaining the recycling rule.

Recycling Game Structure & Level Logic:

To clear a level, the player must process all recycling items in its array.

- Level 1 [Recycling vs General Waste]: 4 items to sort into the Blue Recycling Bin (♻️) or the Black General Waste Bin (🗑️). (Items: 🍏 apple core, 📰 newspaper, 🥫 aluminum can, 🍌 banana peel).

Target score: 4 points.

- Level 2 [Material Streams]: 6 items. Introduces distinct Paper Recycling vs. Plastic Recycling streams. (Items: 🍷 plastic bottle, 📦 cardboard box, 📰 newspaper, 🥤 plastic cup, 🍏 apple core, 🥬 lettuce leaf). Target score: 6 points.

- Level 3 [The Silhouette Peek-a-Boo Game]: 4 standard recycling items + 1 hidden Mystery Recyclable. The final object is locked behind a CSS blur filter (filter: blur(25px)). A large "Reveal 🔍" button drops the blur by 5px per click. 3 large choice buttons appear below; selecting the correct hidden item automatically loads it into the scanner to be sorted. Target score: 5 points.

- Level 4 [The Contamination / Feature Tricking Round]: 6 items. Introduces complex data anomalies where food contamination breaks the recycling process (🍕 greasy pizza box -> General Waste; ☕ paper coffee cup with plastic lining -> General Waste; 🍷 clean, empty plastic water bottle -> Recycling).

Target score: 6 points.

- Level 5 [E-Waste Electronics Class]: 6 items. Introduces electronic waste that requires a separate processing system. A third dedicated yellow "E-Waste" bin button (⚡) dynamically slides into the viewport layout for this level only. Targets: 🔋 battery, 📱 old phone, 📄 paper, 🗑️ can, 🍎 food waste, 💻 old laptop. Target score: 6 points.

Mandatory Logic Validation & Debugging Instructions:

Before generating the final code block, mentally step through the code to verify:

- State Resetting: Ensure that clicking a new level immediately clears the previous level's score, flushes the active arrays, and clears any residual active CSS blur styles.
- Array Length Safeguards: Ensure that the win condition dynamically evaluates `currentScore === currentLevelArray.length` so that the game cannot freeze or become impossible to win.
- Hidden Debug Console: Embed an invisible diagnostic function bound to a small invisible button in the footer labeled "🔧". Tapping this button must print the current application state (active level, active scanner item, category flags, score, and validation status) cleanly into the browser's console log for teacher troubleshooting.

No explanations, no markdown formatting outside the code block, and no placeholders.

P2

Act as an expert frontend game developer and educational math balancer. Write a complete game called "Recycling Sensor Lab" designed for 7-year-old students.

The game must be fully playable, tablet-optimized, and self-contained in ONE code block containing HTML, CSS (in `<style>`), and JavaScript (in `<script>`). Use high-contrast UI metric cards and large touch targets. Clear instructions so users know how to play.

Strict Tablet & Interface Constraints:

1. Pure Tap Interface: Instead of dragging, use a "Tap Item -> Tap Target Property" flow. Interactive button boxes must be at least 85px x 85px with a 20px padding margin to completely eliminate accidental multi-touches on mobile screens.
2. Unlocked Level Architecture: A persistent horizontal level select bar is pinned to the top of the interface (Levels 1 to 5). Open access to any tier must be active.
3. Strict Mathematical Metadata: Every recyclable or waste item emoji must be tied to a rigid JavaScript data object containing true non-visual traits (Weight in grams, Texture type, Material string).

Recycling Game Structure & Level Logic:

Students configure an AI's data sensors to read non-visual metrics of common items passing through the waste facility.

- Level 1 [Weight Metric Engine]: Sort by weight thresholds. Recycling Bins: "Heavy Materials (>500g)" and "Light Materials (<50g)". Items:  heavy wood pallet (800g),  small feather (2g),  heavy metal scrap (600g),  autumn leaf (1g).
- Level 2 [Texture Map Filter]: Match items strictly by surface composition to "Rough" or "Smooth" category sorting plates to separate raw scrap. Items:  concrete block (Rough),  glass screen (Smooth),  organic pine peel (Rough),  polished plastic block (Smooth).
- Level 3 [Material Composition Matrix]: 3 sorting zones appear to separate scrap: [Wood Waste ], [Metal Recycling ], [Plastic Recycling ]. Items:  wooden chair,  iron key,  HDPE bottle,  bamboo chopsticks.
- Level 4 [Clue-Card Inference Matcher]: The AI presents a descriptive text clue card of a recyclable (e.g., "Sensor Data: Weight is 15g, Texture is Smooth, Material is Plastic"). The student must tap the matching object out of 3 visual choices on screen (e.g., choosing  plastic container over  log or  metal key).
- Level 5 [The Double Constraint Sort]: The facility demands precision sorting. The system displays a compound metadata rule (e.g., "Find an item that is BOTH Heavy AND Rough"). The student must successfully identify the target from a changing grid of 6 recycling/waste objects.

Mandatory Logic Validation & Debugging Instructions:

Before generating the final code block, mentally step through the code to verify:

- Value Mismatches: Verify that the conditional logic checking item metadata matches strict string/number types (e.g., no comparison bugs between string "800" and number 800).
- State Cleansing: Ensure that changing levels midway clears the "active object selection" state so inputs do not bleed across levels.
- Embedded Debug Console: Include an explicit console logging mechanism triggered via an invisible footer toggle "". When activated, it must console.log the comparison equations in real-time (e.g., "Checking item:  | Required: Smooth + Plastic | Result: Match") to verify execution runtime accuracy.

No explanations, no markdown formatting outside the code block, and no placeholders.

P3

Act as an expert frontend developer and visual interaction designer. Write a complete game called "Recycling Vision Annotator" designed for 8-year-old students.

The game must be fully playable, tablet-friendly, and self-contained in ONE code block containing HTML, CSS (in <style>), and JavaScript (in <script>). Use CSS layout matrices to simulate high-tech sorting camera target zones. Clear instructions so users know how to play.

Strict Tablet Constraints:

1. Tap-to-Annotate Regions: Instead of requiring students to drag precision mouse vectors to draw bounding boxes, design large, pre-mapped interactive "Hotspot Overlay Zones" directly over target graphics. Tapping a hotspot instantly toggles a bright neon green bounding box line layer over that section. Hotspot targets must be at least 70px wide.
2. Open Level Selection: A persistent dashboard row at the top gives immediate access to Levels 1, 2, 3, 4, and 5 at any point without restricting access.
3. Logical Pass/Fail Verification: Every scene card must mathematically evaluate whether all required features have been successfully annotated before enabling the "Submit to AI Model" action.

Recycling Game Structure & Level Logic:

Students act as human data labelers preparing datasets for a factory computer vision scanner.

- Level 1 [Object Detection / Bounding Boxes]: A conveyor belt full of random items is shown. The prompt instructs: "Draw Bounding Boxes around all Recyclable Containers". The student taps the target recycling objects (🚗 toy car, 🍷 plastic bottle, 🥫 aluminum can) to pop up neon green box borders while ignoring organic items (🍌, 🍎).
- Level 2 [Geometry / Landmark Mapping]: Large geometric packages appear on screen (e.g., 📦 flat cardboard boxes). The task is to "Mark all corners/vertices to detect volume". The student taps the corner points of the boxes to drop distinct crosshair landmark tracking indicators.
- Level 3 [Surface Texture Analysis]: A panel of raw sorting materials appears. The player must differentiate between "Textured/Crinkled Paper" and "Flat/Glossy Plastic" by tapping structural segments of items like 🧻 or 🧴.
- Level 4 [Symbol & Text Tagging]: A shipping container card displays mixed markings. The student must cleanly tap and highlight the exact location of the "Recycling Logo Symbol" (♻️) and the "Tracking Barcode" (||■||), ignoring dirt and scratches.
- Level 5 [Conveyor Scan Challenge]: A rolling queue of 5 scrap objects moves across the viewport. The student has 4 seconds per object to quickly identify and tap its primary visual feature (either its recycling symbol, corner landmark, or material face).

Mandatory Logic Validation & Debugging Instructions:

Before generating the final code block, mentally step through the code to verify:

- DOM Leakage: Confirm that tapping elements inside one level does not trigger event listeners or update variables assigned to other levels.
- Completion Evaluation: Ensure the validation loop relies on array equality verification (`selectedHotspots.sort().toString() === correctHotspots.sort().toString()`) rather than simple numerical array counts, to prevent players from winning by blindly tapping incorrect nodes.
- Hidden Developer Panel: Build a hidden toggle icon "🔧" in the bottom-right corner. When tapped, it must highlight all true correct hotspot bounding zones in a faint yellow border and flash validation flags directly into the console to confirm target click logic works perfectly.

No code truncation or placeholders allowed.

P4

Act as an expert frontend game developer and computer science educator. Write a complete game called "Recycling Prompt Studio" designed for 9-year-old students.

The game must be fully playable, tablet-optimized, and self-contained in ONE code block containing HTML, CSS (in `<style>`), and JavaScript (in `<script>`). Use large, colorful interactive text-chip blocks. Clear instructions so users know how to play.

Strict Tablet Constraints:

1. Keyboardless Interface: Students must assemble complex prompt text strings solely by tapping large, clear word-chip elements. Tapping a chip moves it into the "Active Prompt Construction Bar". No software keyboards are permitted. Chips must have a minimum height of 60px and wrap cleanly using CSS flexbox.
2. Open Access Architecture: A structural header menu lists Levels 1 through 5. Tapping any button instantly changes the active level state without restriction.
3. Deductive Validation Logic: The JavaScript system must check string combinations using strict array matching logic. When a correct combo is compiled, an animated composite graphic component processes on screen.

Recycling Game Structure & Level Logic:

Students prompt an image generation engine to build item feature pairings for recycling manuals.

- Level 1 [Feature Conjunction]: Prompt Goal: "Generate a blue plastic container". Word-chips: [Bottle ], [Can ], [Blue ], [Red ], [Plastic], [Metal]. Correct Combo: [Bottle ] + [Blue ] + [Plastic].
- Level 2 [Shape + Material Balance]: Prompt Goal: "Generate a round container made of metal". Word-chips: [Box ], [Can ], [Metal ], [Paper ]. Correct Combo: [Can ] + [Metal ].
- Level 3 [Riddle-Based Deduction]: The system displays a recycling attribute riddle: "I am a container, I have a clear texture, I am made of plastic, and I hold water." Word chips: [Cardboard Box], [Plastic Bottle ], [Glass Jar]. Selecting [Plastic Bottle ] passes the step.
- Level 4 [Negative Prompting / Exclusions]: Prompt Goal: "Generate an eco-friendly transport tool but EXCLUDE items with wheels". Word-chips: [Train ], [Drone ], [Bicycle ], [Without Wheels ]. Correct Combo: [Drone ] + [Without Wheels ].
- Level 5 [The Triple Attribute Challenge]: A demanding multi-parameter request: "Generate an industrial recycling tool that is waterproof, battery-powered, and made of recycled glass." The student must evaluate 8 shifting word-chips to accurately assemble the 3 core features.

Mandatory Logic Validation & Debugging Instructions:

Before generating the final code block, mentally step through the code to verify:

- Order Independence: Ensure that prompt validation matches elements by checking array containment rather than sequential index positions (e.g., choosing `[Bottle] + [Blue]` must register the same as `[Blue] + [Bottle]`).

- Clear String Flushing: Ensure that shifting levels instantly clears out the "Active Prompt Construction Bar" and flushes all transient tracking values.
- Built-In Debug Logger: Include a diagnostic function linked to a small invisible footer button "🔧". When clicked, it logs the expected chip array vs the user's active chip array directly into the console to debug parsing issues instantly.

Provide ONLY the complete, working HTML code inside a single code block without any truncation.

P5

Act as an expert frontend developer and data science math balancer. Write a complete game called "Recycling Dataset Tycoon" designed for 10-year-old students.

The game must be fully playable, tablet-optimized, and self-contained in ONE code block containing HTML, CSS (in <style>), and JavaScript (in <script>). Use a dark-theme data laboratory dashboard aesthetic. Clear instructions so users know how to play.

Strict Tablet & Mathematical Rules:

1. Massive Counter Controls: Use large, illuminated [+] and [-] button targets (at least 75px x 75px) to allocate data sheets. Do not use sliders or tiny numerical forms.
2. Complete Freedom of Level Access: A permanent row of open navigation buttons at the top switches between Levels 1, 2, 3, 4, and 5 instantly.
3. Authentic Class Balance Formulas: Implement a real mathematical validation script. Accuracy should be calculated dynamically using the variance between categories: $\text{Accuracy} = 100 - (\text{Absolute_Difference} * \text{Weight})$. If data classes are imbalanced, accuracy must mathematically plummet below the passing threshold.

Recycling Game Structure & Level Logic:

Students build clean training datasets to compile a foundational waste classification AI.

- Level 1 [The Class Balance Principle]: Target: Achieve 90%+ accuracy on a Paper (📄) vs Plastic (🗑️) model. Total pool: 20 data slots. The player must use buttons to allocate exactly 10 Paper items and 10 Plastic items. Submitting an imbalanced count (e.g., 18 to 2) triggers a drop in computed accuracy and displays: "Bias Error! Model is over-trained on Plastic and misclassifies paper."
- Level 2 [Cleaning Noisy Input Data]: The data feed is filled with corrupt/glitched recycling images. The student must inspect a pipeline of 10 data cards, tapping "Approve" for clear items and "Trash" for unreadable/blurry items before training the model.
- Level 3 [Multi-Class Distribution Parity]: Train an engine on 3 classes evenly: Metal (🗑️), Glass (🗑️), and Plastic (🗑️). The total data limit is 30 slots. The student must use allocation counters to distribute exactly 10 items into each of the 3 class arrays to achieve a balanced matrix.

- Level 4 [The Human Mislabelled Audit]: Automated web scrapers have tagged materials incorrectly (e.g., a plastic fork labeled as metal). The student must look at a data table grid and tap on the mislabeled cells to correct their feature descriptors prior to compiling.
- Level 5 [The 95% Validation Run]: The ultimate configuration challenge. The student must orchestrate a balanced 40-card dataset spread perfectly across 4 distinct recycling vectors: Paper (📄), Plastic (♻️), Metal (🔧), and Glass (🍷) (10 cards each), hitting exact statistical parity to generate a final Model Validation Score of 95% or higher.

Mandatory Logic Validation & Debugging Instructions:

Before generating the final code block, mentally step through the code to verify:

- Edge Cases: Ensure counter elements never display negative data sizes if a user repeatedly taps the minus button (bound all counters to `Math.max(0, count)``).
- Equation Predictability: Double check the underlying math equation to ensure a perfectly balanced dataset yields **exactly** the specified win percentage required by that level.
- Interactive Debugger HUD: Insert an invisible panel button "🔧" in the console tray. Tapping it overrides and automatically balances all level counts to their winning configuration instantly, allowing teachers to verify level transition logic works accurately.

No comments or explanations outside the block.

P6

Act as an expert frontend developer and data science math instructor. Write a complete game called "Recycling Overfit Doctor" designed for 11-12 year old students.

The game must be fully playable, tablet-focused, and self-contained in ONE code block containing HTML, CSS (in `<style>`), and JavaScript (in `<script>`). Use a stylized data science dashboard featuring real-time updating progress meters representing Training vs Testing Accuracy. Clear instructions so users know how to play.

Strict Tablet & Mathematical Rules:

1. Large Target Stepper Keys: Use massive `[+]` and `[-]` touch containers to adjust parameter integers.
2. Unlocked Navigation Array: A permanent header row provides instant hopping between Levels 1 to 5 at any moment.
3. Mathematical Model Overfitting Core: Write an explicit JavaScript function where:
 - ``Training_Accuracy = Math.min(100, Base + (Epochs * 0.8))``
 - ``Testing_Accuracy = If (Epochs > 30) { Peak_Value - ((Epochs - 30) * 1.2) } Else { Base + (Epochs * 0.7) }``

This cleanly simulates real-world overfitting where excessive training cycles cause testing accuracy to drastically fall while training accuracy reads a perfect 100%.

Recycling Game Structure & Level Logic:

Students tune hyper-parameters to prevent recycling AI models from blindly memorizing data.

- Level 1 [The Epoch Tuner]: An AI model has run for 100 Epochs on green plastic bottles. It has overfit (Training Accuracy: 100%, Testing Accuracy: 40%) and fails to identify a blue bottle. The student must use a [-] touch button to lower the Epoch value down into the sweet spot between 20 and 30. This forces the Training and Testing accuracy bars to balance closely above 85%, clearing the level.
- Level 2 [The Dataset Diversifier]: A model achieves 100% Training Accuracy on a single bottle style but drops to 30% Testing Accuracy in the real plant. The student must tap a button to swap out redundant repeating items ( ,  , ) for alternative material configurations ( ,  , ) until the testing metric crosses 85%.
- Level 3 [The Train/Test Split Balance]: The student manages an incoming pool of 20 recycling data entries. They must allocate them using stepper buttons into a "Training Set Pool" and a "Testing Set Pool". The math requires a balanced 70/30 or 80/20 ratio to avoid overfitting or underfitting.
- Level 4 [Feature Complexity Simplifier]: The network model is processing 12 hyper-complex image features (including conveyor belt background noise and wall shadows). This causes overfitting. The student must toggle specific "Feature Status Switches" to "OFF", removing the irrelevant noise so the AI only tracks core shape and material profiles.
- Level 5 [Live Model Overfit Monitor]: The screen initiates a running recycling stream data simulation with shifting accuracy meters. The student must dynamically adjust training cycles and dataset allocation inputs live on screen to keep Training and Testing scores closely aligned without letting the variance gap widen into an overfit state.

Mandatory Logic Validation & Debugging Instructions:

Before generating the final code block, mentally step through the code to verify:

- Dynamic Curve Syncing: Verify that modifying the Epoch count changes the Training and Testing values instantly via an input event function, rather than requiring a separate manually clicked "refresh" operation.
- Bounds Clamping: Ensure all calculation outputs are clamped strictly between `0` and `100` so progress bars do not break or overshoot the UI grid boundaries.
- Diagnostic Console HUD: Embed an invisible tool switch "" into the footer. Clicking it prints out a comprehensive parameter log into the browser console (e.g., `Epochs: 45, TrainAcc: 98%, TestAcc: 54%, Status: Overfitting`) allowing rapid code debugging.

No explanations, no markdown formatting outside the code block, and no placeholders.

Lesson 2

P1

Act as an expert frontend game developer and early-years UX designer. Write a complete game called "AI Robo-Surveyor" designed for 6-year-old students.

The game must be fully playable, tablet-friendly, and self-contained in ONE code block containing HTML, CSS (in `<style>`), and JavaScript (in `<script>`). Do not use external assets. Use system emojis (, , , ) and large, colorful grid tiles. Clear instructions so users know how to play.

Strict Tablet & Interface Constraints:

1. Tap-to-Reveal Grid: Use a CSS Grid layout with massive cells (minimum 80px x 80px). No tiny buttons.
2. Tool Toggle System: Provide two giant buttons at the bottom: " Scan (Find)" and "drill Drill (Dig)". The student taps a tool to select it, then taps a grid cell to apply it.
3. Unlocked Progression: A top navigation bar displaying "Level 1" to "Level 5" allows instant access to any stage.
4. Safe Fail State: Drilling a hazard does not end the game. It just marks the tile with a "Warning  " icon, shakes the screen gently, and explains "Safety Stop! AI detected a hazard."

Game Structure & Level Logic:

To win, the player must successfully Drill all safe tiles and Flag (Scan) all hazards.

- Level 1 [The Backyard]: 3x3 Grid. Hazards: 2 Rocks (). Safe: Dirt (). Task: Drill the dirt, Scan the rocks.
- Level 2 [The Pipes]: 4x4 Grid. Hazards: 3 Water Pipes (). Introduction of linear hazard patterns.
- Level 3 [No-Build Zones]: 4x4 Grid. Hazards: Buried Dinosaur Bones (). The AI Scanner signals "No-Build" on these tiles.
- Level 4 [The Deep Water]: 5x5 Grid. Hazards: Underground Lake (). Large clusters of hazard tiles.
- Level 5 [Treasure Hunt]: 5x5 Grid. The goal flips. You MUST drill the Artifacts () but avoid the Electric Cables ().

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation & Debugging Instructions:

- Win Condition Check: Ensure the loop checks that EVERY tile is either (Drilled AND Safe) OR (Scanned AND Hazard).
- Touch Event Handling: Use `pointerdown` instead of `click` to ensure instant response on tablets.
- Hidden Debugger: Embed an invisible footer button "🔧". Tapping it reveals the location of all hidden hazards in the console log for teacher verification.

No explanations, no markdown formatting outside the code block, and no placeholders.

P2

Act as an expert frontend game developer and educational math balancer. Write a complete game called "Pulse Scanner AI" designed for 7-year-old students.

The game must be tablet optimised. Use a "Radar/Sonar" aesthetic with high-contrast neon colors. Clear instructions so users know how to play.

Strict Tablet & Interface Constraints:

1. Dual-Mode Inputs: Two massive control buttons: "🐇 Fast Sweep" (Scans 4 tiles at once, 80% accuracy) and "🐢 Deep Pulse" (Scans 1 tile, 100% accuracy).
2. Battery Logic: A "Battery Bar" limits the number of moves. Fast Sweep costs 1 Energy. Deep Pulse costs 3 Energy.
3. Unlocked Levels: Persistent top menu "Level 1-5" for instant access.

Game Structure & Level Logic:

Students must manage energy to map the underground before the battery dies.

- Level 1 [Big Obvious Hazards]: 4x4 Grid. Hazards are large (2x2 size). "Fast Sweep" easily detects them. Battery is high.
- Level 2 [Small Hidden Objects]: 4x4 Grid. Hazards are single tiles (1x1). "Fast Sweep" might show a "Question Mark (?)" on these, requiring a "Deep Pulse" to confirm.
- Level 3 [The Energy Crisis]: 5x5 Grid. Battery is low. Student must use Fast Sweep to clear empty areas and save Deep Pulse only for suspicious spots.
- Level 4 [The Artifact Hunter]: 5x5 Grid. Objective: Find 3 micro-artifacts (👑). Fast Sweep misses them completely (shows "Empty"). Hints are provided ("Artifacts are near rocks").
- Level 5 [Moving Interference]: A "Gopher" (🐹) moves randomly every turn. The student must predict its location and use Deep Pulse to catch it.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.

2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
 3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
 4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
 5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.
- [END VALIDATION BLOCK]

Mandatory Logic Validation & Debugging Instructions:

- Resource Math: Ensure the total Battery Capacity is always sufficient to win if the optimal strategy is used.
- Hitbox Logic: Ensure "Fast Sweep" correctly updates the state of 4 adjacent grid cells simultaneously.
- Hidden Debugger: Invisible "🔧" button logs the exact grid map and battery calculation to the console.

P3

Act as an expert frontend developer and UI designer. Write a complete game called "Subsurface Signal Decoder" designed for 8-year-old students.

The game must be fully playable, tablet-friendly, and self-contained in ONE code block. Use a scientific "Oscilloscope" dashboard layout. Clear instructions so users know how to play.

Strict Tablet Constraints:

1. Tap-to-Target Charts: A large chart area displays signal waves. Below it is a linear grid of buttons (Positions 1-8).
2. Visual Clarity: Signals must be drawn using HTML5 Canvas or simple CSS Div bars. High bars = Strong Signal (Hazard). Low bars = Weak Signal (Safe).
3. Unlocked Levels: Immediate access to Levels 1-5 via top menu.

Game Structure & Level Logic:

Students translate 1D signal data into 2D grid decisions.

- Level 1 [The Clean Signal]: A flat line with one huge spike at Position 4. Student taps Position 4 to drill. Correct!
- Level 2 [Multiple Targets]: Two distinct spikes at Pos 2 and Pos 7. Student must identify both.
- Level 3 [Signal Noise]: The "Safe" line is no longer flat—it's wavy (random height 10-20px). The Hazard spike is much taller (80px). Student must distinguish Signal vs Noise.
- Level 4 [Depth Layers]: Two charts are shown: "Shallow Scan" and "Deep Scan". Hazards only appear on the Deep Scan chart.

- Level 5 [2D Cross-Section]: A 5x5 Grid. Clicking a Row Number displays the signal chart for that specific row. Student must scan row-by-row to find the hidden bunker.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation & Debugging Instructions:

- Threshold Logic: Hardcode the "Hazard Threshold" (e.g., > 60% height) to ensure the game logic never confuses a noisy safe signal with a hazard.
- Canvas Scaling: Ensure the chart renders correctly on different screen widths without blurring.
- Hidden Debugger: "🔧" button highlights the winning coordinates on the UI for rapid testing.

P4

Act as an expert frontend game developer. Write a complete game called "Turbo Scan Surveyor" designed for 9-year-old students.

The game must be tablet optimised. Clear instructions so users know how to play.

Strict Tablet Constraints:

1. Mode Toggles: Large toggle switch at the bottom: "🚀 Turbo (Blurry/Free)" vs "📷 HD (Clear/Costs Time)".
2. Countdown Timer: A prominent visual timer bar. Running out of time resets the level.
3. Unlocked Levels: Top navigation bar for Levels 1-5.

Game Structure & Level Logic:

Students optimize their scan strategy under pressure.

- Level 1 [The Baseline]: 6x6 Grid. 30 Seconds. Plenty of time to click every tile with HD Mode.
- Level 2 [The Speed Run]: 6x6 Grid. 10 Seconds. Impossible to HD scan everything. Must Turbo Scan first, see which tiles are "Darker" (Potential Hazards), then HD scan only those.

- Level 3 [The False Positives]: Turbo Scan results are probabilistic. A "Dark Blob" might be a Rock (Hazard) or just dense Dirt (Safe). HD Scan reveals the truth.
- Level 4 [The Budget Cut]: Time is extended, but "HD Scan" now has a "Cost Counter" (Max 5 HD Scans allowed). Strategy: Turbo everything, HD only the 5 most suspicious spots.
- Level 5 [The Mega Grid]: 8x8 Grid. 15 Seconds. Hazards are connected (Pipes). Student must Turbo scan to find the "Line", then HD scan the start and end points to extrapolate the path.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation & Debugging Instructions:

- Blur Implementation: Use CSS `filter: blur(4px)` or `opacity: 0.5` for the Turbo results to visually simulate low resolution.
- Win Condition: The game checks that all *Hazards* are identified with HD quality. Safe tiles can remain in Turbo quality.
- Hidden Debugger: "🔧" button freezes the timer and reveals the map.

P5

Act as an expert frontend developer and AI curriculum designer. Write a complete game called "AI Texture Trainer" designed for 10-year-old students.

The game must be tablet optimised. Use a "Data Lab" aesthetic. Clear instructions so users know how to play.

Strict Tablet Constraints:

1. Swipe/Tap Labeling: A central card shows a pattern. Large buttons below: "Safe Dirt", "Hard Rock", "Metal Pipe".
2. Instant Feedback Loop: Tapping a label immediately slides the card away and brings a new one (Tinder-style or fast stack).
3. Unlocked Levels: Levels 1-5 accessible immediately.

Game Structure & Level Logic:

Students teach an AI to recognize subsurface textures.

- Level 1 [Distinct Patterns]: "Dirt" is solid brown. "Rock" is grey dots. "Pipe" is silver stripes. 100% visual difference. 10 Cards to sort.
- Level 2 [Similar Patterns]: "Dirt" is dark brown. "Clay" (also safe) is light brown. "Rock" is Grey. Student must group Dirt and Clay as "Safe".
- Level 3 [Noise Injection]: All images have a 20% "static" overlay (CSS pattern). Student must look through the noise to identify the texture.
- Level 4 [Dataset Balancing]: The AI requests a training set. "I need 5 Rocks and 5 Pipes". Student draws cards from a deck and must Accept/Reject to fill the quotas perfectly.
- Level 5 [Confidence Threshold]: The AI makes a guess ("I think this is Rock (75%)"). Student acts as the Human-in-the-Loop to Approve or Correct the AI.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation & Debugging Instructions:

- Array Shuffling: Ensure the deck of cards is randomized every replay so students don't memorize the order.
- Classification Logic: Ensure the underlying ID of the texture (e.g., `type="pipe"`) strictly matches the button logic, regardless of the CSS visual noise.
- Hidden Debugger: "🔧" button auto-completes the current sorting task to test the "Level Complete" animation.

P6

Act as an expert frontend developer and logic puzzle designer. Write a complete game called "Airport AI Security" designed for 11-12 year old students.

The game must be tablet optimised. Use a "X-Ray Monitor" aesthetic (Blue/Orange colors). Clear instructions so users know how to play.

Strict Tablet Constraints:

1. Slider Controls: Large stepper buttons (+ / -) to adjust "AI Sensitivity".
2. Decision Taps: Tap bag to "Flag for Search" or swipe to "Pass".
3. Unlocked Levels: Levels 1-5 accessible immediately.

Game Structure & Level Logic:

Students balance safety (catching threats) vs efficiency (speed).

- Level 1 [Manual Verification]: X-Ray images are blurry. Student has 3 "Manual Search" tokens. 5 Bags pass by. 2 are threats. Student must only search the threats.
- Level 2 [The Sensitivity Slider]: Student sets AI Sensitivity (1-10). Setting it to 10 catches all threats but flags EVERYTHING (False Positives). Setting to 1 misses threats (False Negatives). Goal: Find the balance (Setting 5-6).
- Level 3 [Limited Resources]: 20 Bags. Only 5 Manual Searches allowed. You must tune the AI to flag exactly ~5 suspicious bags. If you flag 10, you run out of searches and lose.
- Level 4 [The Training Loop]: Student manually searches 5 bags to "Teach" the AI. The AI then runs the remaining 15 bags based on that data. If the student mislabeled the training data, the AI fails.
- Level 5 [Total Simulation]: A continuously moving belt. Student adjusts sensitivity slider in real-time. "Threats Missed" causes Game Over. "Passenger Delay" (too many searches) causes Game Over. Keep the needles in the green zone.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation & Debugging Instructions:

- Probability Math: In Level 5, the "Threat" generation must be random but fair (e.g., 20% chance).
- Slider Logic: The Sensitivity formula must be consistent ($\text{DetectionChance} = \text{Sensitivity} * 10\%$).
- Hidden Debugger: "🔧" button reveals the contents of all bags (Threat vs Safe) before they are searched.

Lesson 3

P1

Act as an expert frontend game developer and early-years UX designer. Write a complete game called "AI Sequence Solver" designed for 6-year-old students.

The game must be fully playable, tablet-friendly, and self-contained in ONE code block containing HTML, CSS (in `<style>`), and JavaScript (in `<script>`). Do not use external assets. Use system emojis (, , , ) and large grid tiles. Clear instructions so users know how to play.

Strict Tablet & Interface Constraints:

1. Tap-to-Code: Do NOT use drag-and-drop. Display large Arrow Buttons (   ) at the bottom. Tapping one adds it to a visible "Command Bar" queue.
2. Big "GO" Button: A massive Green Play button executes the sequence step-by-step.
3. Unlocked Progression: Top navigation "Level 1" to "Level 5" for instant access.
4. Clear Instruction Modal: Every level MUST start with a large, friendly popup overlay explaining the goal (e.g., "Help the Robot  get to the Flag !").

Game Structure & Level Logic:

To win, the player must program the exact sequence to get the Bot to the Flag.

- Level 1 [Straight Shot]: 3x3 Grid. Bot at (0,0), Flag at (0,2). Instruction: "Tap Forward  two times."
- Level 2 [The Corner]: 4x4 Grid. Bot needs to move  then . Instruction: "Go straight, then turn down."
- Level 3 [The Obstacle]: 4x4 Grid. A Wall () blocks the direct path. Instruction: "Go around the brick wall!"
- Level 4 [Battery Management]: 5x5 Grid. The sequence is limited to 6 steps maximum (Battery Limit). Instruction: "Find the shortest path!"
- Level 5 [The Recharge]: 5x5 Grid. Path is too long for one charge. Student must route the bot through a Battery () tile. Instruction: "Grab the battery  to keep moving!"

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.

4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
 5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.
- [END VALIDATION BLOCK]

Mandatory Logic Validation & Debugging Instructions:

- Step Delay: Use ``setTimeout`` or ``setInterval`` to allow the student to see the bot move one tile at a time (e.g., 500ms delay), rather than instant teleportation.
- Hidden Debugger: " " button auto-fills the winning sequence for the current level.

No explanations, no markdown formatting outside the code block.

P2

Act as an expert frontend game developer and educational logic designer. Write a complete game called "AI Logic Loop" designed for 7-year-old students.

The game must be tablet optimised. Clear instructions so users know how to play.

Strict Tablet & Interface Constraints:

1. Rule Selectors: Large toggle buttons representing logic states (e.g., "If Path Blocked: Turn Right").
2. Simulation Mode: Student sets the rules, then taps "Activate AI". The bot moves automatically based on the rules.
3. Unlocked Levels: Persistent top menu "Level 1-5".
4. Permanent Objective Bar: A fixed text bar at the top must clearly state the current problem (e.g., "Goal: Avoid the walls!").

Game Structure & Level Logic:

Students program the "Brain" of the bot, not the feet.

- Level 1 [The Straight Line]: Bot faces a wall. Rule needed: "If Path Blocked: Stop". Instruction: "Tell the bot to STOP when it hits a wall."
- Level 2 [The Cornering Bot]: Bot is in a maze corridor. Rule: "If Path Blocked: Turn Right". Instruction: "Help the bot turn the corner."
- Level 3 [The Looper]: Bot is in a square loop. Rule: "If Path Blocked: Turn Left". Instruction: "Make the bot run laps around the square."
- Level 4 [The Sensor Choice]: 2 Sensors available: "Front" and "Right". Student must pick which sensor triggers the turn. Instruction: "Use the FRONT sensor to see the wall."
- Level 5 [Complex Maze]: A zigzag path. Student must combine two rules: "If Front Blocked -> Turn Right" AND "If Right Blocked -> Turn Left". Instruction: "Combine rules to solve the zig-zag!"

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation & Debugging Instructions:

- Loop Prevention: Ensure the game detects infinite loops (e.g., spinning in place) and triggers a "Robot Confused" fail state after 20 moves without progress.
- Hidden Debugger: "🔧" button highlights the tile the bot is currently evaluating logic for.

P3

Act as an expert frontend developer and UI designer. Write a complete game called "Foggy Pathfinder AI" designed for 8-year-old students.

The game must be fully playable, tablet-friendly, and self-contained in ONE code block. Use a "Dark Mode" grid aesthetic. Clear instructions so users know how to play.

Strict Tablet Constraints:

1. Directional Pad: Large D-Pad (Up/Down/Left/Right) for control.
2. Visibility Mask: CSS overlay or class switching to hide tiles > 2 steps away (Greyed out).
3. Unlocked Levels: Immediate access to Levels 1-5.
4. Tutorial Popups: The first time a student encounters Fog or a Ghost, a popup must explain it (e.g., "The Fog hides walls. Move closer to see!").

Game Structure & Level Logic:

Students navigate with incomplete data, updating their plan as new data arrives.

- Level 1 [The Flashlight]: 5x5 Grid. Simple walls. Fog clears permanently once visited (Mapping).

Instruction: "Explore the dark room to find the exit."

- Level 2 [Dead Ends]: 6x6 Grid. Student will encounter a dead end hidden in fog and must backtrack.

Instruction: "If you get stuck, go back and try another way."

- Level 3 [Moving Hazards]: 6x6 Grid. "Ghosts" (👻) patrol. They are only visible when close.

Instruction: "Watch out! Ghosts hide in the fog."

- Level 4 [Sensor Range]: Student gets a "Scan" button (Uses Battery) to briefly reveal the whole map for 2 seconds. Instruction: "Use SCAN to peek at the map!"
- Level 5 [Dynamic Maze]: The walls shift every 5 moves. Instruction: "Be careful! The walls are moving!"

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation & Debugging Instructions:

- Fog Logic: Ensure the fog calculation is based on Manhattan Distance ($\text{Math.abs}(x1-x2) + \text{Math.abs}(y1-y2) \leq 2$).
- Hidden Debugger: "🔧" button toggles "God Mode" (Removes all fog) for testing.

P4

Act as an expert frontend game developer. Write a complete game called "Lidar Mapper Bot" designed for 9-year-old students.

The game must be tablet optimised. Clear instructions so users know how to play.

Strict Tablet Constraints:

1. Tagging Interface: Tap an obstacle to cycle its tag: " ? Unknown" -> "🪑 Furniture" -> "🐶 Pet".
2. Auto-Drive: The bot moves randomly. The student's job is purely data entry/tagging.
3. Unlocked Levels: Top navigation bar for Levels 1-5.
4. Mission Briefing: Each level starts with a text box: "New Room Detected. Help the robot build a map!"

Game Structure & Level Logic:

Students help the AI build a persistent map by filtering out dynamic noise.

- Level 1 [Room Mapping]: Robot bumps into 3 objects. Student tags them as Chairs (🪑). Map is saved. Instruction: "Tap the ? boxes to mark them as Furniture."

- Level 2 [Dynamic Noise]: Robot bumps into a Dog (🐕). Student tags as Pet. The Dog moves. The map should NOT save a wall there. Instruction: "Mark the moving dog as a Pet so the robot ignores it."
- Level 3 [Map Verification]: The Robot tries to return to the Charger. It relies on the student's map. If a Chair was missed, it crashes. Instruction: "Make sure your map is perfect before the battery dies!"
- Level 4 [The Messy Room]: High density of objects. Student must pause the bot to tag everything correctly. Instruction: "Use PAUSE to catch up with the robot."
- Level 5 [Night Mode]: Visuals are dark. Student relies on "Lidar Lines" (Green laser lines) to identify shapes (Boxy = Furniture, Round/Moving = Pet). Instruction: "Use the green lasers to guess the shape."

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation & Debugging Instructions:

- Pathfinding: Implement a basic A* or BFS so the robot can find the charger using the student's generated map.
- Hidden Debugger: "🔧" button reveals the true type of all objects.

P5

Act as an expert frontend developer and AI curriculum designer. Write a complete game called "Efficiency Bot" designed for 10-year-old students.

The game must be tablet optimised. Clear instructions so users know how to play.

Strict Tablet Constraints:

1. Path Drawing: Touch and drag to draw a line path on the grid (Snake style).
2. Metrics HUD: Live display of "Tiles Cleaned: X%" and "Efficiency Score: Y".
3. Unlocked Levels: Levels 1-5 accessible immediately.
4. Goal Overlay: Permanent text at bottom: "Clean 100% of tiles. Don't cross your own line!"

Game Structure & Level Logic:

Students optimize pathing logic to save energy.

- Level 1 [The Sweep]: 4x4 Empty Room. Draw a path that hits every tile without crossing itself. Instruction: "Draw a line that covers every square."
- Level 2 [Islands]: Room has furniture islands. Path must snake around them. Instruction: "Go around the furniture."
- Level 3 [Efficiency]: Cleaning a tile twice costs "Double Energy". Goal: 100% coverage with 0% overlap. Instruction: "Don't visit the same tile twice!"
- Level 4 [Battery Limit]: Total path length is capped. Student cannot waste moves. Instruction: "Shortest path only! Watch your battery."
- Level 5 [The Loop]: Robot must start at the Dock, clean 100%, and end at the Dock. (Hamiltonian Path problem). Instruction: "Clean the room and come back to the charger."

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation & Debugging Instructions:

- Path Validation: Ensure the drawing logic prevents diagonal moves if the game is strictly cardinal (Up/Down/Left/Right).
- Hidden Debugger: "🔧" button instantly draws a valid Hamiltonian path if one exists.

P6

Act as an expert frontend developer and logic puzzle designer. Write a complete game called "Auto-City Sweeper" designed for 11-12 year old students.

The game must be tablet optimised. Clear instructions so users know how to play.

Strict Tablet Constraints:

1. Weight Sliders: Large stepper buttons (+/-) for 3 Parameters: "Speed", "Caution", "Coverage".

2. Simulation View: Top-down city grid with moving cars (🚗) and pedestrians (🚶).
3. Unlocked Levels: Levels 1-5 accessible immediately.
4. Feedback Terminal: A text log that explains *why* a crash happened (e.g., "Crash! Speed was too high for the amount of traffic.").

Game Structure & Level Logic:

Students tune the cost function of the AI to survive complex environments.

- Level 1 [Empty Street]: No traffic. Instruction: "Set SPEED to Max to finish before time runs out."
- Level 2 [Pedestrian Zone]: Lots of walkers. Instruction: "Increase CAUTION to avoid hitting pedestrians."
- Level 3 [Traffic Jam]: Cars move fast. Bot must yield. High "Caution" makes the bot too slow to finish. Instruction: "Balance Speed and Caution to weave through traffic."
- Level 4 [Battery Drain]: High Speed drains battery fast. High Caution drains battery slow (stop/start). Instruction: "Optimize your settings to save battery."
- Level 5 [The Blizzard]: Physics change (Braking distance increases). Student must adjust weights to compensate for slippery roads. Instruction: "It's slippery! Maximize Caution to survive."

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation & Debugging Instructions:

- Collision Logic: Ensure bounding box collision detection is generous enough to not be frustrating, but strict enough to require logic tuning.
- Hidden Debugger: "🔧" button grants infinite battery and invincibility to observe pathing behavior.

Lesson 4

P1

Act as an expert frontend game developer and early-years UX designer. Write a complete game called "Drone Path Trainer" designed for 6-year-old students.

The game must be fully playable, tablet-friendly, and self-contained in ONE code block containing HTML, CSS (in `<style>`), and JavaScript (in `<script>`). Do not use external assets. Use system emojis (🚁, ☁️, 🎯, 🦅) and a sky-blue canvas.

Strict Tablet & Interface Constraints:

1. Touch-to-Draw: Use HTML5 Canvas. Allow the student to use a finger to trace a flight path curve from Start to Finish.
2. Hit Testing: If the drawn line intersects a Cloud (Obstacle), the line turns red and fades (Fail). If it reaches the target safely, the Drone animates along the line (Success).
3. Unlocked Levels: Top navigation "Level 1" to "Level 5" for instant access.
4. Instruction Overlay: Large text popup at start: "Draw a path for the drone! Avoid the clouds!"

Game Structure & Level Logic:

To win, the drawn path must connect Start to End without touching obstacles.

- Level 1 [Clear Sky]: Start (Left) to End (Right). No obstacles. Instruction: "Draw a straight line to the target."
- Level 2 [The Cloud]: One large static Cloud in the middle. Instruction: "Go around the cloud!"
- Level 3 [The Tight Squeeze]: Two clouds with a narrow gap. Instruction: "Careful! Don't touch the sides."
- Level 4 [Moving Birds]: Static obstacles (Birds 🦅). If you get too close, they might scare. Instruction: "Stay away from the birds."
- Level 5 [The Loop]: Target is behind a wall. Student must draw a 'U' shape. Instruction: "Curve around the wall to win!"

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation & Debugging Instructions:

- Canvas Smoothing: Ensure the drawn line uses `lineJoin = 'round'` and `lineCap = 'round'` for smooth tablet drawing.

- Collision Math: Use simple distance checking between points on the line and the center of obstacles.
- Hidden Debugger: "🔧" button auto-draws a winning bezier curve.

No explanations, no markdown formatting outside the code block.

P2

Act as an expert frontend game developer and educational logic designer. Write a complete game called "AI Waypoint Planner" designed for 7-year-old students.

The game must be tablet optimised.

Strict Tablet & Interface Constraints:

1. Tap-to-Place: Tapping the screen drops a numbered Waypoint (1, 2, 3...). The Drone automatically calculates a flight path connecting them.
2. Go Button: A "Launch Drone" button executes the flight.
3. Unlocked Levels: Persistent top menu "Level 1-5".
4. Instruction Bar: Persistent text at top: "Tap to set a path. Avoid the moving clouds!"

Game Structure & Level Logic:

Students plan a path by breaking it into safe segments.

- Level 1 [Connect the Dots]: 3 Stars appear. Place a waypoint on each star to collect them.
- Level 2 [Static Obstacles]: Walls block the direct path. Student must place a waypoint at the corner to guide the drone around.
- Level 3 [Moving Clouds]: Clouds drift slowly left/right. Student must time the flight or route far around them.
- Level 4 [The Zig-Zag]: Narrow corridor. Student must place 4 precise waypoints to navigate the tunnel without hitting walls.
- Level 5 [Efficiency]: Limited to 3 Waypoints max. Must find the optimal triangle path to avoid a massive storm.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation & Debugging Instructions:

- Path Logic: Ensure the drone moves at a constant speed between points using linear interpolation.
- Hidden Debugger: "🔧" button reveals the hitboxes of all obstacles.

P3

Act as an expert frontend developer and physics simulation designer. Write a complete game called "Drone Velocity Lab" designed for 8-year-old students.

The game must be fully playable, tablet-friendly, and self-contained in ONE code block.

Strict Tablet Constraints:

1. Slider Input: Massive slider at the bottom: "Speed: Slow (Precise) <---> Fast (Drifty)".
2. Simulation View: Top-down view of a drone flying a track.
3. Unlocked Levels: Immediate access to Levels 1-5.
4. Instruction Modal: Pop-up at start: "Adjust the Speed Slider. Slow down for sharp turns!"

Game Structure & Level Logic:

Students learn that "Maximum Speed" isn't always the best solution.

- Level 1 [The Straightaway]: A straight line. Optimization: Set Speed to MAX (100%).
- Level 2 [The Gentle Curve]: A wide oval track. Optimization: Set Speed to High (80%).
- Level 3 [The Hairpin]: A sharp U-turn. High speed causes the drone to drift into the wall. Optimization: Low Speed (30%).
- Level 4 [Variable Track]: Alternating straights and turns. Student must actively adjust the slider *during* the flight.
- Level 5 [Wind Shear]: Crosswinds push the drone. Student must increase speed to fight the wind, but decrease speed to make the turns. Balancing act.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation & Debugging Instructions:

- Physics Engine: Implement simple inertia. $\text{Velocity} = \text{Velocity} + (\text{Target} - \text{Current}) * \text{TurningFactor}$. High speed reduces TurningFactor .
- Hidden Debugger: "🔧" button shows the predicted flight path line based on current speed.

P4

Act as an expert frontend game developer. Write a complete game called "Adaptive Drone Logic" designed for 9-year-old students.

The game must be tablet optimised.

Strict Tablet Constraints:

1. Zone Tapping: The map is divided into colored grid sectors. Tapping a sector cycles its rule: "Green (Fast)" -> "Yellow (Medium)" -> "Red (Slow)".
2. Execute Button: "Run Flight Path".
3. Unlocked Levels: Top navigation bar for Levels 1-5.
4. Feedback Text: If crash: "Crashed in City Zone! Speed was too high."

Game Structure & Level Logic:

Students program the behavior logic before the flight begins.

- Level 1 [Basic Zones]: Map has "Open Sky" (Safe for Fast) and "Buildings" (Needs Slow). Instruction: "Fly Slow near buildings, Fast in the sky."
- Level 2 [Battery Saver]: Long distance. "Slow" drains too much battery (time in air). Must use "Fast" in open areas to make it in time.
- Level 3 [Turbulence]: "Mountain Zone". Needs "Medium" speed. Fast crashes (instability), Slow crashes (stalls).
- Level 4 [The Mixed Map]: Complex grid of City, Mountains, and Ocean. Student must correctly tag all 3 types.
- Level 5 [Hidden Rules]: Student isn't told the rules. Must run a test flight, see where it crashes, adjust the zone logic, and retry (Debugging).

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.

4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
 5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.
- [END VALIDATION BLOCK]

Mandatory Logic Validation & Debugging Instructions:

- Win Logic: Ensure battery constraints and speed constraints don't conflict (e.g., impossible to finish).
- Hidden Debugger: "🔧" button auto-sets the correct rules for all zones.

P5

Act as an expert frontend developer and AI curriculum designer. Write a complete game called "Drone Cost Commander" designed for 10-year-old students.

The game must be tablet optimised.

Strict Tablet Constraints:

1. Node Interface: Circles (Nodes) connected by lines (Edges). Edges display a number (Cost).
2. Tap-to-Route: Tap a Node to extend the path to it.
3. Live Calculator: HUD shows "Current Total Cost: 15".
4. Unlocked Levels: Levels 1-5 accessible immediately.
5. Goal Text: "Connect Start to Finish. Max Cost allowed: [X]."

Game Structure & Level Logic:

Students act as the A* Algorithm, finding the optimal route.

- Level 1 [Math Warmup]: Two paths. Top path costs $5+5=10$. Bottom path costs $2+2=4$. Instruction: "Choose the path with the smallest numbers."
- Level 2 [The Trap]: The visually shorter path has high costs (Mountains = 50). The visually longer path has low costs (Valley = 10). Instruction: "Don't be fooled! Shortest distance isn't always cheapest."
- Level 3 [Battery Budget]: Max Cost is 20. Student must experiment to find a route under 20.
- Level 4 [Multi-Stop]: Must visit a "Package Pickup" node before the "Finish" node. Optimization becomes harder.
- Level 5 [Dynamic Costs]: Weather changes! "Headwind" adds +10 to all West-facing edges. Student must recalculate the best route based on new data.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.

2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
 3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
 4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
 5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.
- [END VALIDATION BLOCK]

Mandatory Logic Validation & Debugging Instructions:

- Graph Logic: Ensure graph is directed or undirected consistent with the visual arrows.
- Hidden Debugger: "🔧" button highlights the optimal path in green.

P6

Act as an expert frontend developer and logic puzzle designer. Write a complete game called "Drone Script Architect" designed for 11-12 year old students.

The game must be tablet optimised.

Strict Tablet Constraints:

1. Block Interface: Large colored buttons for commands: [ Move], [ Turn], [ Loop], [ If Blocked].
2. Timeline Bar: Tapping a block appends it to the bottom "Script" bar. Tapping a block in the script removes it.
3. Unlocked Levels: Levels 1-5 accessible immediately.
4. Error Logs: If the drone fails, show "Runtime Error: Hit wall at step 4."

Game Structure & Level Logic:

Students engage in algorithmic thinking and debugging.

- Level 1 [Sequential]: Move 3, Turn Right, Move 2. Instruction: "Program a simple delivery route."
- Level 2 [Loops]: Route is 20 steps long but repeats. Use a [ Loop 4x] block to save space.
Instruction: "Use a Loop to repeat actions!"
- Level 3 [Obstacles]: Static walls. Instruction: "Navigate the maze."
- Level 4 [Conditionals]: Dynamic gate. Use [ If Blocked] -> [Fly Over]. Instruction: "Use IF blocks to handle the closed gate."
- Level 5 [The Round Trip]: Deliver package A, pick up package B, return to Hub. Requires nested logic or long sequences.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation & Debugging Instructions:

- Execution Engine: Ensure the code executes asynchronously (step-by-step animation) so the student can debug.
- Hidden Debugger: "🔧" button instant-completes the level.

Lesson 5

P1

Act as an expert frontend game developer and early-years UX designer. Develop a functional prototype game called "Drone Ducklings" designed for 6-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 (Canvas) + JavaScript.
- Target Hardware: Tablet devices (Touch & Drag).
- UI/UX: Large, bright sprites. Immediate feedback.
- Pedagogical Target: Teach "Leader-Follower" behavior—if the leader moves too fast or turns too sharp, the followers (swarm) get confused.
- Difficulty Scaling: 5 Levels. Open access to all levels.
- Instructions: Clear text overlay at the start of each level (e.g., "Drag the Mama Drone slowly!").

GAMEPLAY & AI LOGIC

- Level 1 [The Tether]: One "Mama Drone" (Leader) and 3 "Baby Drones" (Followers). The babies follow the leader's *past position* (Snake logic), not the current position.
- Level 2 [The Turn]: A sharp corner. If the student drags the leader too fast around the corner, the babies clip the wall.

- Level 3 [The Crowd]: 5 Babies. Navigating through pillars. The swarm flows around obstacles like a liquid.
- Level 4 [Signal Loss]: Moving too fast turns the connecting line RED. If the line breaks, a baby gets lost. Student must stop and wait.
- Level 5 [The Herd]: 10 Babies. A winding path. Requires slow, deliberate movement to keep the swarm together.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Check for variable scope issues in the animation loop.
2. LOGIC CHECK: Confirm collision detection applies to Followers, not just the Leader.
3. NUMERICAL AUDIT: Verify that the "Catch-up Speed" variable is realistic (not too slow).
4. SIMULATION RUN: Simulate a touch event dragging the leader into a wall to test fail states.
5. REFACTOR: Correct any logic that prevents the game from resetting properly.

[END VALIDATION BLOCK]

P2

Act as an expert frontend game developer. Develop a functional prototype game called "Swarm Safety Shield" designed for 7-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 + JavaScript.
- Target Hardware: Tablet devices (Multi-touch buttons).
- UI/UX: Two massive buttons at the bottom: "● Expand (Safe)" and "● Squeeze (Risky)".
- Pedagogical Target: Teach the trade-off between "Safety" (Spacing out) and "Utility" (Fitting through gaps).
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Squeeze to fit! Expand to stay safe!"

GAMEPLAY & AI LOGIC

- Level 1 [Wide Gates]: Large openings. Student learns that "Expand" keeps drones safe from bumping each other.
- Level 2 [Narrow Pipes]: Tight tunnels. Student must "Squeeze" to fit, then immediately "Expand" after exiting to prevent overheating.
- Level 3 [The Rhythm]: Alternating Wide -> Narrow -> Wide gates.
- Level 4 [Moving Walls]: The tunnel walls breathe in and out. Dynamic adjustment required.
- Level 5 [The Debris Field]: Small asteroids. "Expand" activates a force field that pushes debris away, but "Squeeze" is needed for the solid gates.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Check for infinite loops in the repulsion calculation.
2. LOGIC CHECK: Ensure drones push away from *each other* and *walls* correctly.
3. NUMERICAL AUDIT: Verify repulsion force constants prevent overlapping sprites.
4. SIMULATION RUN: Simulate a "Squeeze" action for 10 seconds to ensure physics don't explode.
5. REFACTOR: Fix any jittering movement artifacts.

[END VALIDATION BLOCK]

P3

Act as an expert frontend developer. Develop a functional prototype game called "Drone Shepherd AI" designed for 8-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 (Canvas) + JavaScript.
- Target Hardware: Tablet devices (Drag and Drop).
- UI/UX: Sidebar with infinite "Green Beacons" and "Red Jammers".
- Pedagogical Target: Teach indirect control—guiding agents by modifying the environment (Attraction/Repulsion) rather than steering them.
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Green pulls them closer. Red pushes them away."

GAMEPLAY & AI LOGIC

- Level 1 [The Magnet]: Swarm is stuck in a corner. Place a Green Beacon to pull them to the exit.
- Level 2 [The Shield]: Swarm is flying towards a storm. Place a Red Jammer to deflect them.
- Level 3 [The Breadcrumbs]: Create a curved path using multiple Green Beacons to guide them through a maze.
- Level 4 [Dynamic Hazards]: Storms move. Student must move the Jammers in real-time to protect the swarm.
- Level 5 [Split & Merge]: Use a Jammer to split the swarm around a central pillar, then a Beacon to merge them back.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Verify event listeners for drag-and-drop on touch devices.
2. LOGIC CHECK: Ensure attraction/repulsion forces decay over distance (Inverse Square Law).
3. NUMERICAL AUDIT: Check that forces are balanced so drones don't orbit beacons forever.
4. SIMULATION RUN: Simulate placing 10 beacons to check performance limits.

5. REFACTOR: Optimize the physics loop if frame rate drops.
[END VALIDATION BLOCK]

P4

Act as an expert frontend developer and simulation designer. Develop a functional prototype game called "Flock Master Algorithm" designed for 9-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 + JavaScript.
- Target Hardware: Tablet devices (Sliders).
- UI/UX: Three clear sliders: "Separation", "Alignment", "Cohesion". Visual simulation of 50 drones.
- Pedagogical Target: Teach the three core rules of flocking behavior (Reynolds' Boids).
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Adjust the sliders to match the Goal behavior!"

GAMEPLAY & AI LOGIC

- Level 1 [Social Distancing]: Goal: "Spread out!". Solution: High Separation, Low Cohesion.
- Level 2 [The Tight Ball]: Goal: "Form a cluster!". Solution: High Cohesion, Low Separation.
- Level 3 [The Stream]: Goal: "Fly in the same direction!". Solution: High Alignment.
- Level 4 [Fluid Flow]: Goal: "Flow around obstacles". Solution: Balanced Separation and Alignment.
- Level 5 [Predator Reaction]: A 'Hawk' attacks. Goal: "Dodge and Regroup". Solution: High Separation (Agility) + High Cohesion (Safety).

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Check Boids algorithm implementation for correct vector math.
2. LOGIC CHECK: Ensure boundary wrapping (screen edges) works so drones don't vanish.
3. NUMERICAL AUDIT: validate slider ranges (0.0 to 1.0) map effectively to physics forces.
4. SIMULATION RUN: Max out all sliders to check for physics instability/explosions.
5. REFACTOR: Clamp velocity variables to prevent teleportation bugs.

[END VALIDATION BLOCK]

P5

Act as an expert frontend developer and logic designer. Develop a functional prototype game called "Agri-Swarm Optimizer" designed for 10-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 + JavaScript.
- Target Hardware: Tablet devices (Tap interface).
- UI/UX: Grid map of farm fields. Battery indicators on drones.
- Pedagogical Target: Teach efficiency and local optimization—sending the nearest resource saves energy (Cost Functions).
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Tap yellow fields to water them. Pick the closest drone to save battery!"

GAMEPLAY & AI LOGIC

- Level 1 [Proximity]: 1 Drone, 3 Fields. Always pick the closest field first.
- Level 2 [Parallelism]: 2 Drones. Split the map visually (Left vs Right) to maximize speed.
- Level 3 [Refueling]: Drones must return to dock. Plan a loop: Field -> Field -> Dock.
- Level 4 [Capacity Limit]: Drones can only water 2 fields before empty. Don't send a half-empty drone to a far field.
- Level 5 [The Heatwave]: Rapid requests. Student must prioritize "Red" (Critical) fields over "Yellow" fields.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Check for memory leaks in the timer loop.
2. LOGIC CHECK: Ensure drones return to availability pool after completing a task.
3. NUMERICAL AUDIT: Verify battery drain rates allow for a perfect solution margin.
4. SIMULATION RUN: Simulate assigning all drones to one field to check queue logic.
5. REFACTOR: Ensure the "Game Over" state triggers correctly when crops die.

[END VALIDATION BLOCK]

P6

Act as an expert frontend developer and puzzle designer. Develop a functional prototype game called "Swarm Traffic Controller" designed for 11-12 year old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 + JavaScript.
- Target Hardware: Tablet devices (Drag-and-drop Logic Blocks).
- UI/UX: Top-down traffic view. Logic slots for North, South, East, West.
- Pedagogical Target: Teach decentralized logic and priority rules (e.g., FIFO, Weighted Priority).
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Drag the Rules to the Controller to stop the crashes!"

GAMEPLAY & AI LOGIC

- Level 1 [The Stop Sign]: North/South traffic only. Rule: "North Stop. South Go."
- Level 2 [FIFO]: 4-way intersection. Rule: "If Intersection Empty -> Go. Else -> Wait." (First In, First Out).
- Level 3 [Priority Classes]: Ambulance Drones (Red) vs Delivery Drones (Blue). Rule: "Blue Yields to Red."
- Level 4 [The Roundabout]: Rule: "Yield to drones inside the circle."
- Level 5 [The Optimization]: High traffic volume. Student must adjust the "Green Light Duration" timer variable to let batches of drones through without backing up.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Check syntax of the rule evaluation engine.
2. LOGIC CHECK: Confirm that "Yield" logic actually stops the sprite movement.
3. NUMERICAL AUDIT: Verify collision bounding boxes are precise.
4. SIMULATION RUN: Spawn 20 drones at once to stress-test the collision logic.
5. REFACTOR: Add visual indicators (Red/Green lines) to show which drones are allowed to move.

[END VALIDATION BLOCK]

Lesson 6

P1

Act as an expert frontend game developer and early-years UX designer. Develop a functional prototype game called "Robo-Recycler Loop" designed for 6-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 (Canvas) + JavaScript.
- Target Hardware: Tablet devices (Tap-to-Move).
- UI/UX: Big, bright nodes. Green = Start/End. Blue = Trash.
- Pedagogical Target: Teach "Loop Closure"—the path isn't finished until the bot returns to the start.
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Tap the Trash  to pick it up, then go back to the Charger ⚡!"

GAMEPLAY & AI LOGIC

- Level 1 [The Circle]: 3 stops arranged in a circle. Start -> A -> B -> Start.
- Level 2 [The Line]: Stops are in a line. Student must go out and come back (A -> B -> C -> B -> A -> Start).
- Level 3 [The Fork]: Two paths. Go left to get trash, return to center, go right to get trash, return to start.
- Level 4 [Battery Check]: A battery bar drains with every step. If the loop is too long or inefficient, the bot dies before reaching home.
- Level 5 [The One-Way]: Arrows on the floor. Student must follow the arrows to close the loop.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Check for infinite loops in the node connection logic.
2. LOGIC CHECK: Ensure the win condition only triggers if *all* trash is collected AND bot is at Start.
3. NUMERICAL AUDIT: Verify battery levels allow for small mistakes but punish inefficient routing.
4. SIMULATION RUN: Simulate a player forgetting to return home to test the "Incomplete" state.
5. REFACTOR: Add visual arrows showing the path drawn so far.

[END VALIDATION BLOCK]

P2

Act as an expert frontend game developer. Develop a functional prototype game called "One-Way City AI" designed for 7-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 + JavaScript.
- Target Hardware: Tablet devices (Touch path drawing).
- UI/UX: Grid map with arrows on the streets.
- Pedagogical Target: Teach "Directed Graphs"—following constraints to form a valid path.
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Follow the arrows! Connect all the houses and get back to the Garage."

GAMEPLAY & AI LOGIC

- Level 1 [The Square]: A simple clockwise loop. Counter-clockwise is blocked.
- Level 2 [The Figure 8]: A central intersection. Student must cross it twice in the correct direction.
- Level 3 [Dead Ends]: Some streets lead to nowhere. Student must plan ahead to avoid getting stuck.
- Level 4 [Switching Directions]: A traffic light changes the direction of one street every 5 seconds. Timing required.
- Level 5 [The Maze]: A complex grid of one-way streets. Requires finding the specific "Hamiltonian Path" that visits every node once.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Verify the graph traversal algorithm handles directed edges.
2. LOGIC CHECK: Ensure the player cannot draw a line against the arrow direction.
3. NUMERICAL AUDIT: Verify that at least one valid solution exists for every level.
4. SIMULATION RUN: Simulate a path entering a dead end to ensure "Stuck" state triggers clearly.
5. REFACTOR: Highlight valid next moves in Green to reduce frustration.

[END VALIDATION BLOCK]

P3

Act as an expert frontend developer. Develop a functional prototype game called "Fuel Saver AI" designed for 8-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 (Canvas) + JavaScript.
- Target Hardware: Tablet devices (Drag to connect nodes).
- UI/UX: Map with House Nodes. A large "Fuel Tank" gauge.
- Pedagogical Target: Teach "Cost Functions"—the goal isn't just to finish, but to finish with the *most fuel remaining*.
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Connect the houses! Use the Shortest Path to save fuel!"

GAMEPLAY & AI LOGIC

- Level 1 [The Triangle]: 3 Houses. A-B-C is short. A-C-B is long. Student watches fuel bar drop differently.
- Level 2 [The Zig-Zag]: Houses arranged in two rows. Zig-zagging is expensive. Going down one row and up the other is cheap.
- Level 3 [The Hub]: A central warehouse. It's cheaper to return to the hub to reload than to drive between distant houses.
- Level 4 [Terrain Costs]: Muddy roads cost 2x Fuel. Student must route around the mud even if the path looks longer visually.
- Level 5 [The Budget]: "Max Fuel: 50". Student must try multiple combinations until they find the one path under 50.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Check distance calculation logic (Euclidean vs Manhattan).
2. LOGIC CHECK: Ensure terrain modifiers (mud) correctly multiply the edge cost.

3. NUMERICAL AUDIT: Verify level budgets are tight enough to require thought.
 4. SIMULATION RUN: Calculate all permutations to ensure the budget isn't impossible.
 5. REFACTOR: Show a "Ghost Path" of the previous attempt for comparison.
- [END VALIDATION BLOCK]

P4

Act as an expert frontend developer and simulation designer. Develop a functional prototype game called "TSP Challenge: Human vs Bot" designed for 9-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 + JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: A map with random points. "Draw Path" mode and "AI Reveal" button.
- Pedagogical Target: Introduce the Traveling Salesman Problem. Compare human intuition vs machine precision.
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Visit every star! Keep the path short. Then tap 'AI Solve' to see the perfect path."

GAMEPLAY & AI LOGIC

- Level 1 [5 Stars]: Simple polygon shape. Human usually gets it right. AI gets same score.
- Level 2 [10 Stars]: Complex shape. Human lines often cross (inefficient). AI uncrosses them (2-opt algorithm).
- Level 3 [Clustered]: Stars grouped in cities. Human tends to jump between clusters randomly. AI finishes one cluster before moving to the next.
- Level 4 [The Crossing]: A figure-8 layout. AI shows how to avoid crossing the path to save distance.
- Level 5 [Speed Round]: 15 Stars. 30 seconds. Human panics and draws messy lines. AI solves it instantly. Visual comparison of "Your Score: 500m" vs "AI Score: 420m".

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Implement a basic Nearest Neighbor or 2-Opt algorithm for the AI solver.
2. LOGIC CHECK: Ensure the AI path visits every node exactly once.
3. NUMERICAL AUDIT: Verify the distance calculation is pixel-perfect.
4. SIMULATION RUN: Run the AI solver on Level 5 to ensure it finishes < 100ms.
5. REFACTOR: Add a visual overlay comparing the two paths (Red line vs Blue line).

[END VALIDATION BLOCK]

P5

Act as an expert frontend developer and logic designer. Develop a functional prototype game called "Heavy Hauler Logic" designed for 10-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 + JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Truck icon with a "Weight Bar". Bins with numbers (e.g., "50kg").
- Pedagogical Target: Teach dynamic cost functions—the cost of moving changes based on the truck's current state.
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Heavy trucks burn more gas! Plan your route to stay light as long as possible."

GAMEPLAY & AI LOGIC

- Logic Rule: $\text{FuelCost} = \text{Distance} * (1 + \text{CurrentWeight})$.
- Level 1 [Light vs Heavy]: Two items. One is 10kg, one is 100kg. Pick up 100kg *last* to save fuel on the approach.
- Level 2 [Capacity Cap]: Truck Max Load = 100kg. Bins are 60kg and 50kg. Impossible to take both. Must return to depot to unload in between.
- Level 3 [The Hill]: Driving up hill costs 3x fuel if heavy. Route around the hill or empty the truck first.
- Level 4 [Route Sorting]: 5 items of varying weights. Optimal strategy: Sort by weight ascending (heuristic).
- Level 5 [Strict Limit]: Fuel is extremely tight. Only the mathematically perfect order (collecting lightest to heaviest close to depot) works.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Check the fuel consumption formula implementation.
2. LOGIC CHECK: Ensure the truck weight updates immediately upon pickup.
3. NUMERICAL AUDIT: Verify that Level 5 is mathematically solvable.
4. SIMULATION RUN: Simulate a "Heavy First" strategy vs "Light First" strategy to prove the difference.
5. REFACTOR: Display floating text "-X Fuel" after every move to show cost.

[END VALIDATION BLOCK]

P6

Act as an expert frontend developer and puzzle designer. Develop a functional prototype game called "Fleet Logistics Commander" designed for 11-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 + JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Split screen: Map View (Top) and Timeline View (Bottom). 3 colored trucks.
- Pedagogical Target: Teach "Clustering"—grouping nearby points together to be served by a single agent.
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Draw loops to group the houses! Assign one truck to each group."

GAMEPLAY & AI LOGIC

- Level 1 [Visual Clustering]: 10 houses. 5 on Left, 5 on Right. Student draws a circle around the Left group (Assign Truck A) and Right group (Assign Truck B).
- Level 2 [Timeline conflict]: Truck A has too many stops. It finishes late. Student must move some stops to Truck B to balance the load.
- Level 3 [Overlap]: Houses are mixed. Student must use a "K-Means" style logic to find the best dividing line.
- Level 4 [Battery Range]: Trucks have limited range. Groups must be tight and circular. Long linear groups fail.
- Level 5 [AI Optimization]: Student creates a plan. Then "AI Optimize" runs. The game overlays the AI's clusters vs the Student's clusters to show efficiency gains.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Check for syntax errors in the clustering visualization.
2. LOGIC CHECK: Ensure houses cannot be assigned to two trucks at once.
3. NUMERICAL AUDIT: Verify time calculations ($\text{Time} = \text{Distance} / \text{Speed} + \text{ServiceTime}$).
4. SIMULATION RUN: Assign all houses to Truck A to test "Overload" failure state.
5. REFACTOR: Add a "Total Fleet Time" metric to score the player.

[END VALIDATION BLOCK]

Lesson 7

P1

Act as an expert frontend game developer and early-years UX designer. Develop a functional prototype game called "Auto-Bus Sorter" designed for 6-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 (Canvas) + JavaScript.
- Target Hardware: Tablet devices (Drag and Drop).
- UI/UX: Big, bright Passengers (Red, Blue, Green) and Buses. Large "Rule Blocks".
- Pedagogical Target: Teach Classification—grouping items by attribute.
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Drag the passengers to the matching bus!"

GAMEPLAY & AI LOGIC

- Level 1 [Manual Sort]: 3 Passengers (Red, Blue, Green). 3 Buses. Student drags them manually.
- Level 2 [The Gate]: Passengers walk on a conveyor. Student taps a "Switch" to send Red left and Blue right.
- Level 3 [Rule Building]: Student places a tile: "IF [Red Person] -> [Open Red Door]". The game runs automatically based on the rule.
- Level 4 [Capacity]: Bus holds only 2. Student must use a rule: "IF [Bus Full] -> [Call New Bus]".
- Level 5 [The Mix]: Passengers have two colors (Red Hat, Blue Shirt). Rule: "Sort by HAT color".

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Check for infinite loops in the conveyor logic.
2. LOGIC CHECK: Ensure sorting rules trigger exactly when the passenger passes the sensor.
3. NUMERICAL AUDIT: Verify bus capacity counters update correctly.
4. SIMULATION RUN: Simulate a stream of 10 mixed passengers to test the sorting logic.
5. REFACTOR: Add visual confetti when a bus leaves fully loaded.

[END VALIDATION BLOCK]

P2

Act as an expert frontend game developer. Develop a functional prototype game called "Robo-Bus Scheduler" designed for 7-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 + JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: A timeline bar with drag-and-drop "Event Pins". Digital Clock display.
- Pedagogical Target: Teach Temporal Logic (Scheduling).
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Drag the 'GO' pin to 8:00 AM!"

GAMEPLAY & AI LOGIC

- Level 1 [The Alarm]: Bus is sleeping. Goal: "Start Engine at 7:00". Student drags the Start Pin to 7:00.
- Level 2 [The Route]: Bus needs to be at School by 8:00. It takes 30 mins. Student must calculate backwards and set Start to 7:30.
- Level 3 [Conditional Stop]: "IF [Stop has People] -> [Wait 5 mins]". Student toggles the logic switch.
- Level 4 [The Loop]: Bus needs to repeat the route every hour. Use a "Loop 4x" block.
- Level 5 [Sync]: Two buses. Bus A arrives at 9:00. Bus B must leave at 9:05. Student syncs the timelines.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Check timeline calculation logic for negative durations.
2. LOGIC CHECK: Ensure "Wait" commands actually pause the bus sprite.
3. NUMERICAL AUDIT: Verify that time intervals (e.g., 30 mins) scale correctly to screen pixels.
4. SIMULATION RUN: Run the schedule at 10x speed to check for "Late Arrival" fail states.
5. REFACTOR: Add a "Fast Forward" button for testing.

[END VALIDATION BLOCK]

P3

Act as an expert frontend developer. Develop a functional prototype game called "Heatmap Transit AI" designed for 8-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 (Canvas) + JavaScript.
- Target Hardware: Tablet devices (Tap to Deploy).
- UI/UX: City map overlay. Red blobs = High demand. Green blobs = Low demand.
- Pedagogical Target: Teach Data-Driven Decision Making (reacting to sensor data).
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Tap the Garage to send buses. Target the Red blobs!"

GAMEPLAY & AI LOGIC

- Level 1 [Reading Data]: One big red blob. Send 3 buses there. Easy win.
- Level 2 [Supply & Demand]: Limited buses (5). Two red blobs. Split the fleet 2 and 3.
- Level 3 [Dynamic Shift]: Morning Rush (East side is red) -> Evening Rush (West side is red). Student must re-route buses mid-day.
- Level 4 [Efficiency]: Don't send empty buses to Green zones. It wastes money.
- Level 5 [The Surge]: Sudden spike in demand (Stadium Event). Student must deploy the "Reserve Fleet" instantly.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Check heatmap rendering logic (gradient overlay).
2. LOGIC CHECK: Ensure passengers decrease when a bus arrives (Satisfying demand).
3. NUMERICAL AUDIT: Verify that 1 Bus = X Passengers capacity.
4. SIMULATION RUN: Simulate a "Zero Bus" scenario to ensure the Crowd Anger meter fills up.
5. REFACTOR: Add a "Crowd Anger" bar that rises if wait times are too long.

[END VALIDATION BLOCK]

P4

Act as an expert frontend developer. Develop a functional prototype game called "Transit Oracle AI" designed for 9-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 + JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Split screen: Live Map (Top) and Time-Series Graph (Bottom).
- Pedagogical Target: Teach Prediction—acting on future probability, not just current state.
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "The line is going up! Send buses NOW before the station gets full!"

GAMEPLAY & AI LOGIC

- Level 1 [Pattern Recognition]: Graph goes up every day at 8am. It is 7am. Dispatch now!
- Level 2 [Lag Time]: It takes 10 seconds for a bus to arrive. Student must click dispatch 10s *before* the spike on the graph.
- Level 3 [Event Logic]: "Calendar: Concert Tonight". Student infers high demand at the Stadium stop.
- Level 4 [Weather Data]: "Rain Forecast". Rain = More Bus Riders (People don't walk). Increase fleet size by 20%.
- Level 5 [Algorithm Tuning]: Set the "Threshold". "Auto-Dispatch if probability > 80%". Watch the AI play itself.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Check graph drawing code for canvas pixel alignment.
2. LOGIC CHECK: Ensure "Early Dispatch" rewards points, "Late Dispatch" loses points.
3. NUMERICAL AUDIT: Verify travel time delay is consistent.
4. SIMULATION RUN: Simulate a perfect prediction to verify Win State is reachable.
5. REFACTOR: Add a "Forecast Line" (dotted line) showing the AI's guess.

[END VALIDATION BLOCK]

P5

Act as an expert frontend developer and logic designer. Develop a functional prototype game called "Transit Sync Master" designed for 10-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 + JavaScript.
- Target Hardware: Tablet devices (Sliders).
- UI/UX: Metro map style. Nodes are Transfer Stations. Sliders control "Departure Offset".
- Pedagogical Target: Teach Synchronization—optimizing connections in a network.
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Adjust the timing slider! Don't let passengers miss their connection!"

GAMEPLAY & AI LOGIC

- Level 1 [The Handoff]: Bus A arrives at :00. Bus B leaves at :05. Perfect transfer. If Bus B leaves at :59, passengers wait 1 hour (Fail).
- Level 2 [The Intersection]: Two lines cross. Adjust speeds so they don't block each other at the junction.
- Level 3 [Frequency]: High Frequency (Every 5 mins) vs Low Frequency (Every 30 mins). Connect a feeder line to a main line.
- Level 4 [The Hub]: 4 lines meet at Central Station. Align all 4 sliders to meet at the "Pulse" (Top of the hour).
- Level 5 [Ripple Effects]: One bus breaks down. Adjust the schedule of the *other* lines to compensate (Dynamic Rescheduling).

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Check modulo arithmetic for time looping (60 mins).
2. LOGIC CHECK: Ensure "Missed Connection" logic calculates wait time correctly.
3. NUMERICAL AUDIT: Verify passenger happiness score decays with wait time.
4. SIMULATION RUN: Randomly jitter the sliders to test robustness.
5. REFACTOR: Add "Passenger Emojis" jumping from one bus to another to show success.

[END VALIDATION BLOCK]

P6

Act as an expert frontend developer and puzzle designer. Develop a functional prototype game called "Live Transit Commander" designed for 11-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 + JavaScript.
- Target Hardware: Tablet devices (Drag-and-Drop, Path Drawing).
- UI/UX: Live City Map with moving dots. "Alert Feed" sidebar.
- Pedagogical Target: Teach Fault Tolerance—keeping a system running when parts fail.
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Fix the problems! Drag reserve buses to cover gaps. Draw new routes around traffic!"

GAMEPLAY & AI LOGIC

- Level 1 [Breakdown]: A bus turns Black (Broken). Drag a "Reserve Bus" from the depot to that location to pick up passengers.
- Level 2 [Traffic Jam]: A street turns Red (Slow). Draw a detour path for Line 5 to avoid the jam.
- Level 3 [Bunching]: Two buses are too close together. Command the second bus to "Hold" for 2 minutes to restore spacing (Headway management).
- Level 4 [The Roadblock]: Main street closed. Redesign the entire route network using the remaining streets.
- Level 5 [AI Assist]: The AI suggests 3 solutions to a crisis. Student must analyze and pick the best one (Human-in-the-loop).

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Check pathfinding (A*) for the detour drawing tool.
2. LOGIC CHECK: Ensure dragged buses snap to the nearest valid route.
3. NUMERICAL AUDIT: Verify "System Health" percentage calculation.
4. SIMULATION RUN: Trigger 3 disasters at once to ensure the UI doesn't crash.
5. REFACTOR: Add visual "Repair Crews" that fix broken buses over time.

[END VALIDATION BLOCK]

Lesson 8

Act as an expert frontend game developer and early-years UX designer. Develop a functional prototype game called "Robo-Triage Trainer" designed for 6-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 (Canvas) + JavaScript.
- Target Hardware: Tablet devices (Drag and Drop).
- UI/UX: Large Patient Sprites with tags. A friendly Robot character observing.
- Pedagogical Target: Teach ****Supervised Learning****—showing examples to an AI so it learns the category.
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Drag the patient to the matching room. Teach the robot!"

GAMEPLAY & AI LOGIC

- Level 1 [Color Training]: Red Tag -> Red Room. Blue Tag -> Blue Room. Robot says "I am learning: Red = Red!"
- Level 2 [Shape Training]: Tags have shapes (Triangle, Circle). Robot learns to sort by shape.
- Level 3 [The Test]: The Robot tries to sort one itself. It holds a Red tag and asks "Does this go to Red Room?" Student taps "Yes" or "No" (Reinforcement).
- Level 4 [Feature Confusion]: Patients have Red Hats but Blue Tags. Rule: "Sort by TAG, not Hat." Teach the AI to ignore the hat.
- Level 5 [Automation]: The Robot runs fast. Student watches and only taps "Stop" if the AI makes a mistake.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Check drag event listeners for touch compatibility.
2. LOGIC CHECK: Ensure "Feature Confusion" level ignores the wrong attribute (Hat).
3. NUMERICAL AUDIT: Verify the Robot's auto-sort speed is tracking correctly.
4. SIMULATION RUN: Simulate a player correcting the robot to check score updates.
5. REFACTOR: Add a "Brain Bar" that fills up as the AI learns.

[END VALIDATION BLOCK]

P2

Act as an expert frontend game developer. Develop a functional prototype game called "Predictive Pharmacy" designed for 7-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 + JavaScript.
- Target Hardware: Tablet devices.

- UI/UX: "AI Forecast Screen" with % bars. Shelf slots.
- Pedagogical Target: Teach ****Probabilistic Prediction****—making decisions based on likelihood.
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Read the chance percentage! Stock the shelves."

GAMEPLAY & AI LOGIC

- Level 1 [Certainty]: AI says "100% Chance of Rain". Stock Umbrellas.
- Level 2 [High Probability]: AI says "90% Chance of Flu". Stock Medicine. (Ignore the 10% chance of Sunburn).
- Level 3 [Uncertainty]: AI says "50% Cold, 50% Hot". Student must split the stock evenly.
- Level 4 [Risk Management]: AI says "10% Chance of Zombie Virus". It's low, but the risk is high. Student must stock at least 1 Zombie Kit just in case.
- Level 5 [Training Data]: The AI makes a bad guess. Student feeds it "Yesterday's Weather" to improve the prediction for tomorrow.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Check probability logic (Random number generation).
2. LOGIC CHECK: Ensure inventory limits are strictly enforced.
3. NUMERICAL AUDIT: Verify that high-probability events occur more often in simulation.
4. SIMULATION RUN: Simulate stocking 0 items for a 100% event to test Fail State.
5. REFACTOR: Add a visual "Confidence Meter" to the AI's text.

[END VALIDATION BLOCK]

P3

Act as an expert frontend developer. Develop a functional prototype game called "Ambulance Path Weights" designed for 8-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 (Canvas) + JavaScript.
- Target Hardware: Tablet devices (Tap/Paint interface).
- UI/UX: City grid. Nodes and Edges. Ambulance icon.
- Pedagogical Target: Teach ****Pathfinding Weights****—how AI finds routes based on numerical cost, not visual distance.
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Lower the road cost to guide the ambulance!"

GAMEPLAY & AI LOGIC

- Level 1 [Cost Basics]: Two paths. Short path has "Traffic Cost: 50". Long path has "Traffic Cost: 10". AI picks the Long path. Student paints the Short path Green (Cost: 1) to force the AI to switch.
- Level 2 [Obstacles]: A broken bridge has "Cost: Infinity". AI is stuck. Student paints a detour road.
- Level 3 [Multi-Agent]: Two ambulances. Student must paint separate lanes so they don't block each other.
- Level 4 [Dynamic Weights]: Traffic changes randomly. Student must update weights in real-time to keep the AI flow efficient.
- Level 5 [The Algorithm]: Show the "Search Nodes" (A* visualization). Student sees how the AI "thinks" by exploring low-cost nodes first.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Implement a basic Dijkstra or A* algorithm.
2. LOGIC CHECK: Ensure changing edge weights triggers an immediate path recalculation.
3. NUMERICAL AUDIT: Verify that "Infinity" cost effectively blocks the path.
4. SIMULATION RUN: Simulate a map where target is unreachable to check error handling.
5. REFACTOR: Visualize the path as a glowing line.

[END VALIDATION BLOCK]

P4

Act as an expert frontend developer. Develop a functional prototype game called "Hospital Logic Scheduler" designed for 9-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 + JavaScript.
- Target Hardware: Tablet devices (Drag and Drop blocks).
- UI/UX: Calendar Grid. Staff Blocks (Dr. A, Nurse B). "Constraint List" sidebar.
- Pedagogical Target: Teach **Constraint Satisfaction**—solving a puzzle where rules limit valid moves.
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Drag staff to the slots. Don't break the red rules!"

GAMEPLAY & AI LOGIC

- Level 1 [Hard Constraints]: Rule: "Dr. Green cannot work Mondays." If student drags Dr. Green to Mon, the block bounces back Red.
- Level 2 [adjacency Constraints]: Rule: "Dr. Blue and Dr. Red cannot work the same shift." (Personality clash).
- Level 3 [Minimums]: Rule: "Every shift needs at least 1 Nurse."
- Level 4 [Fatigue Logic]: Rule: "No one can work 2 shifts in a row."

- Level 5 [Optimization]: "Find a schedule that satisfies ALL rules with the fewest staff." (CSP Solver logic).

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Check logic for constraint evaluation functions.
2. LOGIC CHECK: Ensure all constraints are checked on every drop event.
3. NUMERICAL AUDIT: Verify that a valid solution exists for every level.
4. SIMULATION RUN: Brute-force check possible combinations to ensure solvability.
5. REFACTOR: Add a "Check Validity" button that highlights errors.

[END VALIDATION BLOCK]

P5

Act as an expert frontend developer and logic designer. Develop a functional prototype game called "Cluster Clinic AI" designed for 10-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 + JavaScript.
- Target Hardware: Tablet devices (Tap to Build).
- UI/UX: Map with hundreds of "Person Dots". Draggable "Hospital Centroids".
- Pedagogical Target: Teach **Clustering Algorithms (K-Means)**—grouping data points around a center.
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Drag the Hospital to the center of the crowd to minimize travel time."

GAMEPLAY & AI LOGIC

- Level 1 [One Cluster]: A cloud of points. Drag the hospital until the "Total Distance" score is minimized (The Centroid).
- Level 2 [Two Clusters]: Two distinct crowds. You have 2 Hospitals. If you put both in one crowd, the other crowd turns Red (Too far). Split them up!
- Level 3 [Outliers]: A few people live far away. AI lesson: "Should you move the whole hospital for one person?" (Overfitting vs Generalization).
- Level 4 [K-Means Iteration]: Press "Auto-Adjust". Watch the hospitals slide automatically to the mathematical center. Student must manually refine the result.
- Level 5 [Weighted Clusters]: Some dots are "Critical Patients" (Weight = 10). The Hospital must be pulled closer to them than regular patients.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Implement basic Euclidean distance sum calculation.
2. LOGIC CHECK: Ensure lines are drawn from each Person to their *nearest* Hospital.
3. NUMERICAL AUDIT: Verify that the "Total Distance" score updates in real-time.
4. SIMULATION RUN: Simulate random placement vs optimal placement to set Star Rating thresholds.
5. REFACTOR: Visual lines connecting people to hospitals should update instantly on drag.

[END VALIDATION BLOCK]

P6

Act as an expert frontend developer and puzzle designer. Develop a functional prototype game called "Utility Grid AI" designed for 11-year-old students.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5 + JavaScript.
- Target Hardware: Tablet devices (Sliders).
- UI/UX: City Grid. Sliders for "Priority Weights".
- Pedagogical Target: Teach **Utility Functions**—how AI decides what to sacrifice by calculating value.
- Difficulty Scaling: 5 Levels. Open access.
- Instructions: "Set the Priority Sliders. The AI will switch off the lowest priority zones."

GAMEPLAY & AI LOGIC

- Level 1 [Basic Values]: Slider: Hospital Priority (High) vs Park Priority (Low). AI cuts Park power. Success.
- Level 2 [Conflicting Goals]: Heatwave. AC is needed everywhere. Student must tune weights: "Saving Lives > Comfort".
- Level 3 [The Threshold]: Grid Capacity is 50%. Student must assign numerical values so that exactly 50% of load (the least valuable) is shed.
- Level 4 [Dynamic Utility]: Time changes (Night vs Day). Malls are useless at night (Utility = 0). Student programs time-based weights.
- Level 5 [Objective Function]: Student writes a logic formula: $\text{Score} = (\text{Health} * 2) + (\text{Economy} * 1)$. The AI executes the cuts to maximize this Score.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Check power summation and utility sorting logic.
2. LOGIC CHECK: Ensure the AI *always* cuts the lowest utility node first.
3. NUMERICAL AUDIT: Verify that "Critical" nodes (Hospitals) stay on if grid capacity allows.
4. SIMULATION RUN: Max out all weights to see if the AI fails gracefully (Random cut vs Error).

5. REFACTOR: Add a "Utility Score" readout to show the mathematical result.
[END VALIDATION BLOCK]

Lesson 9

P1

Act as an expert frontend game developer and early-years UX designer. Write a complete game called "Smart Pipe Bot" designed for 6-year-old students.

The game must be fully playable and functional as a standalone HTML5 application (HTML/CSS/JS). Do not use external assets. Use system emojis (🤖, 💧, 🏠, 🚧) and a clean water-blue canvas.

Strict Tablet & Interface Constraints:

1. Trace-to-Connect: Use HTML5 Canvas. The student traces a line with their finger to guide the "Smart Bot" 🤖 from the Water Tower to the House.
2. Hit Testing: If the path hits a Rock (🚧), the pipe breaks. If it reaches the house, water flows.
3. Unlocked Levels: Top navigation "Level 1" to "Level 5" for instant access.

Game Structure & Level Logic (Teaching AI Pathfinding):

The goal is to teach that AI must "look ahead" to avoid obstacles.

- Level 1 [Direct Path]: A straight line. Instruction: "Guide the Bot to the house!"
- Level 2 [Obstacle Avoidance]: A rock blocks the straight path. Instruction: "AI connects the dots! Go around the rock."
- Level 3 [The Leak Fix]: A broken pipe icon (💧) is on screen. Instruction: "The Bot detects a leak! Draw a path through it to fix it."
- Level 4 [Complex Routing]: Multiple rocks. Instruction: "Find the clear path!"
- Level 5 [Branching Logic]: Connect one tower to TWO houses. Instruction: "AI Networks! Feed both houses."

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants use valid values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Canvas: Use `lineCap = 'round'` for smooth drawing.
- Debugger: "🔧" button auto-draws the winning path.

P2

Act as an expert frontend game developer and educational designer. Write a complete game called "Flow Logic Master" designed for 7-year-old students.

The game must be fully playable and functional as a standalone HTML5 application. Use large UI elements and a dashboard aesthetic.

Strict Tablet & Interface Constraints:

1. Tap-to-Adjust: Giant [+] and [-] buttons to control Valve Openness (0 to 10).
2. Visual Feedback: A "Pressure Gauge" bar. Green Zone is 4-6. Red is too high/low.
3. Unlocked Levels: Top navigation "Level 1" to "Level 5".

Game Structure & Level Logic (Teaching Control Systems):

The goal is to teach how systems "sense" errors and "correct" them.

- Level 1 [Manual Control]: Pressure is low (2). Instruction: "Pressure too low! Tap [+] to reach the Green Zone."
- Level 2 [The Overshoot]: Pressure is critical (9)! Instruction: "Too high! Tap [-] to lower it."
- Level 3 [System Delay]: Buttons react with a 1-second delay. Instruction: "Wait for the sensor! Don't click too fast."
- Level 4 [Automated Limit]: Set a "Safety Limit" slider. Instruction: "Teach the AI! Set the max limit to 6 to stop the flow automatically."
- Level 5 [Balancing Act]: Two gauges, one valve. Instruction: "Find the sweet spot that makes BOTH gauges green."

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants use valid values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: ``Pressure = (ValveValue * 0.8) + BaseLine``.
- Debugger: "🔧" button instantly sets ValveValue to the winning number.

P3

Act as an expert frontend game developer and data science instructor. Write a complete game called "Sonic Leak Hunter" designed for 8-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. Sensor Grid: A 3x3 grid of "City Blocks". Tapping a block reveals a "Sound Wave" graph.
2. Pattern Matching: Students must identify if the wave looks "Smooth" (Safe) or "Jagged" (Leak).
3. Unlocked Levels: Top navigation "Level 1" to "Level 5".

Game Structure & Level Logic (Teaching Classification):

The goal is to teach how AI tells the difference between "Good" and "Bad" data.

- Level 1 [Training Data]: Shown two waves side-by-side. Instruction: "Train the AI! Spiky waves mean leaks. Tap the spiky one."
- Level 2 [The Scan]: 3 Grid blocks. 1 contains a hidden leak. Instruction: "Scan the city. Find the spiky wave pattern."
- Level 3 [Noise Filtering]: Waves are moving. Instruction: "Ignore the small wiggles. Find the BIG spikes."
- Level 4 [Setting Thresholds]: A slider called "Sensitivity". Instruction: "Adjust the AI filter until it ONLY outlines the jagged wave in red."
- Level 5 [Batch Processing]: 6 blocks. Instruction: "Deploy the AI Scanner to find all 3 leaks at once."

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants use valid values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Visualization: Use HTML Canvas to draw sine waves. ``LeakWave`` should add visible noise to the curve.

- Debugger: "🔧" highlights the correct leak grid cells.

P4

Act as an expert frontend game developer and math instructor. Write a complete game called "Future Flow Predictor" designed for 9-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. Input Sliders: Large sliders for "Temperature" (☀️) and "Population" (👥).
2. Output Graph: A dynamic bar chart showing "Predicted Water Usage".
3. Target Match: A dashed line shows the "Actual Demand".

Game Structure & Level Logic (Teaching Prediction):

The goal is to teach that inputs (variables) allow us to predict outputs.

- Level 1 [One Variable]: Population is fixed. Instruction: "Hotter days mean more water. Move the Temp slider to match the predicted line."
- Level 2 [Two Variables]: Temp is fixed. Instruction: "More people mean more water. Adjust the Population slider."
- Level 3 [The Algorithm]: A "Multiplier" button (1x, 2x, 3x). Instruction: "The algorithm is under-guessing. Increase the weight multiplier to 2x."
- Level 4 [Training Data]: Show a graph of Yesterday (High usage). Instruction: "Today looks like Yesterday. Match the pump to yesterday's level."
- Level 5 [Logic Puzzle]: High Temp + Low Population. Instruction: "Balance the inputs to predict the exact water need."

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants use valid values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{Demand} = (\text{Temp} * 2) + (\text{Population} * 1.5)$.
- Debugger: "🔧" prints the exact target integer to the console.

P5

Act as an expert frontend game developer and logic instructor. Write a complete game called "Smart Grid Scheduler" designed for 10-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. Timeline UI: A visual 24-hour bar (Day vs Night).
2. Draggable Blocks: "Pump Tasks" that consume energy.
3. Cost Meter: A "Total Cost" display. (Night energy is cheaper).

Game Structure & Level Logic (Teaching Optimization):

The goal is to teach how AI solves "Scheduling Problems" to save money.

- Level 1 [Cost Functions]: Instruction: "Running pumps at Night costs \$2. Day costs \$10. Drag the block to Night to save money."
- Level 2 [Constraints]: Instruction: "The pipe only fits 3 tasks at once. Arrange them so they don't overlap."
- Level 3 [Safety Constraints]: Instruction: "Keep the Reservoir > 50%. Don't delay the pump too long or the tank runs dry!"
- Level 4 [Opportunity Windows]: "Solar Power" is free at Noon. Instruction: "Move the biggest task to the Solar window."
- Level 5 [Complex Optimization]: 5 Tasks. Instruction: "Optimize the schedule: Lowest Cost + No Empty Tank."

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants use valid values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Constraint Check: Ensure user cannot stack overlapping blocks beyond array capacity.
- Debugger: "🔧" auto-sorts blocks into the optimal time slots.

P6

Act as an expert frontend developer and data science math instructor. Write a complete game called "City Aqua-Mind" designed for 11-12 year old students.

The game must be fully playable and functional as a standalone HTML5 application. Use a stylized data science dashboard.

Strict Tablet & Mathematical Rules:

1. Dashboard UI: Real-time graphs for "Reservoir Level", "City Satisfaction", and "Illegal Usage".
2. Unlocked Navigation: Permanent header row for Levels 1-5.
3. Mathematical Core: $\text{Satisfaction} = 100 - (\text{Restrictions} * 15) - (\text{Unmet_Demand} * 2)$.

Game Structure & Level Logic (Teaching Anomaly Detection):

The goal is to teach how AI spots "Weird Data" (Anomalies) in a complex system.

- Level 1 [Establishing Baseline]: Instruction: "Watch the graph. Establish the normal usage pattern (40-60)."
- Level 2 [Spike Detection]: Data jumps to 90. Instruction: "Anomaly detected! That spike isn't normal usage. Click it to flag a pipe burst."
- Level 3 [Pattern Recognition]: Instruction: "Find the 'Thief' Signature: High Usage + Low Pressure. Flag the house matching this pattern."
- Level 4 [False Positives]: Usage is high, but it's a holiday. Instruction: "Context matters. Don't flag this spike, it's just a holiday party."
- Level 5 [Homeostasis]: A live simulation. Instruction: "Balance the system. Detect 3 anomalies while keeping Satisfaction > 50%."

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants use valid values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Dynamic Syncing: Verify sliders update the Satisfaction Score instantly.
- Diagnostic Console: " " prints `Inflow`, `Usage`, `Sat\_Score` to console.

Lesson 10

P1

Act as an expert frontend game developer and early-years UX designer. Write a complete game called "Power Path Tracer" designed for 6-year-old students.

The game must be fully playable and functional as a standalone HTML5 application. Do not use external assets. Use system emojis (⚡, 🔋, 🏠, ☁️, 🤖) and a dark mode canvas.

Strict Tablet & Interface Constraints:

1. Drag-to-Wire: Use HTML5 Canvas. The student drags a "Wire" from the Power Source to the Device.
2. Physics Logic: "Voltage Drop". If the wire is too long, the electricity fades (turns yellow -> red) and the device won't start.
3. Edge Solution: Place a "Local Battery" (Edge Node) closer to the device to fix the distance problem.
4. Unlocked Levels: Top navigation "Level 1" to "Level 5".

Game Structure & Level Logic (Teaching Edge Computing):

- Level 1 [Short Circuit]: Device is close. Draw a short line. (Easy success).
- Level 2 [The Long Haul]: Device is far away. Drawing a long line causes the power to fade before reaching the end. (Fail state introduction).
- Level 3 [Edge Node]: A "Mini Power Box" 🔋 appears near the device. Connect Main -> Mini Box -> Device. (Success).
- Level 4 [Multiple Devices]: Two houses. One is close (direct wire), one is far (needs Mini Box).
- Level 5 [Cloud vs Edge]: A "Cloud Server" (Far) and "Edge Robot" (Near). Show that the Near connection is faster (turns green instantly) while Cloud is slower.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify wire length calculations use valid pixels.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Visuals: Use `strokeStyle` gradients or colors to visualize power loss over distance.
- Debugger: "🔧" button auto-completes the wiring.

P2

Act as an expert frontend game developer and educational designer. Write a complete game called "Grid Balance Hero" designed for 7-year-old students.

The game must be fully playable and functional as a standalone HTML5 application. Use large UI elements (Sliders/Scales).

Strict Tablet & Interface Constraints:

1. The Scale UI: A visual balance scale or gauge. Left Side = "City Lights" (). Right Side = "AI Training" (.
2. Input: A slider to allocate power. Left for People, Right for Robots.
3. Win Condition: Keep the needle in the Green Zone for 10 seconds.

Game Structure & Level Logic (Teaching Load Balancing):

- Level 1 [Normal Day]: Low demand. Keep the slider in the middle.
- Level 2 [The Spike]: Suddenly, the AI Robot grows (Training Cycle). It sucks power! Slide right to feed it, but not too much!
- Level 3 [The Cold Night]: Houses need heat! Slide left. The AI must wait.
- Level 4 [Priorities]: The AI demands power, but the City is blinking red. Instruction: "People first!" Limit the AI power.
- Level 5 [The Storm]: Both sides fluctuate wildly. Rapidly adjust the slider to prevent a blackout.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify slider values (0-100) match the gauge logic.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: ``GridStability = 100 - Math.abs(Supply - Demand)``.
- Debugger: " " locks the slider to the perfect balance point.

P3

Act as an expert frontend game developer and logic instructor. Write a complete game called "Cool Grid Architect" designed for 8-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. Drag-and-Drop Map: A map with zones: River (💧), Snowy Mountain (❄️), Desert (☀️), City (🏙️).
2. Item Drawer: Placeable items: "AI Data Center", "Wind Turbine", "Solar Panel".
3. Feedback: Placing items shows an efficiency score (e.g., Data Center on Mountain = +10 Cool Bonus).

Game Structure & Level Logic (Teaching Infrastructure Optimization):

- Level 1 [Heat Management]: Place the Data Center. If placed in Desert, it overheats (Fail). Place in Snow (Win).
- Level 2 [Hydro Power]: Connect the Data Center to energy. Place a Turbine in the River for max power.
- Level 3 [Distance Cost]: Place the Data Center. It needs to be cool (Mountain) but close enough to the City to work. Find the middle ground.
- Level 4 [Green Energy]: The City needs power. Use Solar Panels (Desert) and Wind (Mountain) to power it without using Coal.
- Level 5 [Master Plan]: Place 2 Data Centers and 3 Power Sources. Maximize the "Eco-Score" (Low Heat + High Power).

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify collision detection and scoring math.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Logic: ``Score = CoolingFactor + PowerOutput - DistancePenalty``.
- Debugger: "🔧" auto-places items in optimal coordinates.

P4

Act as an expert frontend game developer and math instructor. Write a complete game called "Auto-Power Manager" designed for 9-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. Resource Dashboard: Bars for "Battery Charge" (🔋), "Fossil Fuel" (💧), "AI Compute Speed" (🚀).
2. The Toggle: A switch between "Manual Mode" and "AI Auto Mode".
3. Goal: Keep the lights on without running out of fuel.

Game Structure & Level Logic (Teaching Automation):

- Level 1 [Manual Battery]: Sun goes down. Solar stops. Tap "Use Battery" to survive the night.
- Level 2 [Fossil Backup]: Battery dies. Tap "Burn Fuel" (but watch the pollution meter!).
- Level 3 [Throttling]: Fuel is low. Drag "AI Speed" down to save power. (Sacrifice compute for survival).
- Level 4 [Training the Bot]: Play manually for 10 seconds. The game records your moves. Then activate "Auto Mode" and watch the ghost-cursor mimic you.
- Level 5 [The Stress Test]: A storm hits (Low Solar). Activate your trained Auto-Bot. If you taught it well (to throttle AI and use battery), it survives.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify resource depletion rates are realistic.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{Energy_Consumption} = \text{BaseLoad} + (\text{AI_Speed} * 2)$.
- Debugger: "🔧" refills all resource bars.

P5

Act as an expert frontend game developer and logic instructor. Write a complete game called "Power Market Tycoon" designed for 10-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. Market UI: A "Price Slider" (\$/kWh).
2. Buyers: Animated icons representing "Citizens" (😊) and "Mega-Corp AI" (🤖).
3. Supply Bar: Total available electricity.

Game Structure & Level Logic (Teaching Market Dynamics):

- Level 1 [Supply & Demand]: Low Supply. Raise the price to prevent running out of power.
- Level 2 [The Monopoly]: A "Mega-Corp AI" enters. It has infinite money. If price is low, it buys 100% of the power. Citizens black out. Raise price to stop it!
- Level 3 [The Cap]: You cannot raise prices too high, or Citizens go broke. Find the "Fair Price" window.
- Level 4 [Variable Rates]: Set two prices: "Day Rate" and "Night Rate". Push the AI to buy at night (cheaper) to save Day power for people.
- Level 5 [Regulation Crisis]: Energy Deficit. Set a strict "Max Buy Limit" on the AI while managing dynamic pricing to keep the grid solvent.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify currency and logic math.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Logic: $\text{AiDemand} = \max(0, 100 - \text{Price} * \text{Elasticity})$.
- Debugger: "🔧" sets the optimal price point.

P6

Act as an expert frontend developer and data science instructor. Write a complete game called "Grid Orchestrator" designed for 11-12 year old students.

The game must be fully playable and functional as a standalone HTML5 application. Use a "Mission Control" aesthetic.

Strict Tablet & Mathematical Rules:

1. Network Graph: Nodes representing Hospital (🏥), Traffic Lights (🚦), School (🎓), and AI Server Farm (🖥️).
2. Power Status: A falling percentage meter (100% -> 0%).
3. Action: Tap nodes to "Cut Power" (Load Shedding) to save the remaining energy for critical nodes.

Game Structure & Level Logic (Teaching Critical Heuristics):

- Level 1 [Triage]: Power drops to 80%. Instruction: "Cut the AI Server Farm to save the Hospital."
- Level 2 [Cascading Failure]: The grid is overloaded. If you don't cut 2 sectors in 5 seconds, the whole grid blows.
- Level 3 [Automated Rules]: Write a logic rule: "IF Power < 50%, THEN Cut [Select Sector]". Test if your rule saves the city.
- Level 4 [Rerouting]: A main line breaks. Toggle switches to reroute power from the Industrial Zone to the Residential Zone.
- Level 5 [Blackout Simulation]: Survive 60 seconds of chaotic failure. You must manually load-shed non-essentials while keeping the Hospital and Traffic Lights online > 90% of the time.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify power drain rates and time limits.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Scoring: Critical Nodes = 1000pts. Non-Critical = 100pts.
- Debugger: "🔧" freezes the power drain.

Lesson 11

P1

Act as an expert frontend game developer and early-years UX designer. Write a complete game called "AI Spy Cam" designed for 6-year-old students.

The game must be fully playable and functional as a standalone HTML5 application. Do not use external assets. Use system emojis (🧸, ⚽, 🧢, 🐶, 🌳) and a camera viewfinder UI.

Strict Tablet & Interface Constraints:

1. Viewfinder: A central box showing the "Target Object" (e.g., A Red Ball).
2. The Crowd: A grid of random emojis.
3. Interaction: Tap the items in the crowd that match the Target.
4. AI Training: A "Train" button. When tapped, the AI learns a new object to look for.

Game Structure & Level Logic (Teaching Object Recognition):

- Level 1 [Exact Match]: Target is a 🏀. Find the 3 🏀s in the crowd.
- Level 2 [Similarity]: Target is a "Red Ball". Find things that look similar (Red Apples 🍎, Red Balloons 🎈).
- Level 3 [Training Mode]: The AI doesn't know what a "Bear" is. Tap 3 Bears 🐻 to "Train" the model. Now the AI highlights them automatically.
- Level 4 [Lost Item]: Find the "Blue Hat" hidden behind a tree. (Partial occlusion).
- Level 5 [The Scanner]: The scene is moving (scrolling). Tap the target object before it leaves the screen.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify grid generation logic.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Visuals: Use a "Scanning" animation (green box overlay) when a match is found.
- Debugger: "🔧" auto-clicks all correct targets.

P2

Act as an expert frontend game developer and educational designer. Write a complete game called "Depth Vision Bot" designed for 7-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. Dual View: Screen split in half. Top = "Normal Camera" (Color). Bottom = "Depth Map" (Grayscale: White=Close, Black=Far).
2. Safety Zone: A "Clearance Bar". Objects that are too "White" (Close/Tall) trigger a warning.
3. Interaction: Tap objects in the Depth Map to measure their distance.

Game Structure & Level Logic (Teaching Computer Vision Depth):

- Level 1 [Dark vs Light]: Instruction: "In AI Vision, Light means Close, Dark means Far. Tap the Closest object (Whitest)."
- Level 2 [The Bridge]: A truck is approaching a bridge. Look at the Depth Map. If the Truck is "Whiter" (Taller) than the Bridge, hit STOP!
- Level 3 [Buffer Zone]: Set a "Safety Distance". Drag a slider to turn everything within 5 meters Red.
- Level 4 [Fog Mode]: The Color camera is foggy (can't see). Use ONLY the Depth Map to guide a car around obstacles.
- Level 5 [3D Scanner]: Tap 3 objects to "Ping" them and build a simple 3D wireframe mesh (visualized by drawing connecting lines).

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify color-to-distance mapping logic.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: `Brightness = 255 - (Distance * 10)`.
- Debugger: "🔧" toggles the Depth View on/off.

P3

Act as an expert frontend game developer and logic instructor. Write a complete game called "Edge Mode Detective" designed for 8-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. The Toggle: A switch between "RGB Mode" (Real photo style) and "Edge Mode" (Black background, Green outlines).
2. The Counter: A digital counter to tally objects.
3. Visual Noise: The RGB mode has shadows, sun glare, and confusing colors. Edge Mode removes this.

Game Structure & Level Logic (Teaching Feature Extraction):

- Level 1 [Visual Noise]: Instruction: "Count the pedestrians. It's too dark! Switch to 'Edge Mode' to see their outlines clearly."

- Level 2 [Overlap]: People are standing close together. In Color, they blend in. In Edge Mode, you see distinct lines. Count them: 5.
- Level 3 [Traffic Count]: Cars are moving fast with headlights (glare). Edge Mode strips the glare. Count the cars.
- Level 4 [The Filter]: Adjust "Line Thickness". Thin lines show too much detail (leaves). Thick lines show only the Cars. Find the right setting.
- Level 5 [Speed Count]: 10 objects pass by. You must toggle modes instantly to verify the count.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify counting logic matches displayed sprites.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Visuals: Use HTML Canvas `stroke()` to simulate edge detection wireframes.
- Debugger: "🔧" displays the correct count in the console.

P4

Act as an expert frontend game developer and math instructor. Write a complete game called "Drone Pixel Painter" designed for 9-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. Paint Palette: Colors representing classes: Blue (Water), Grey (Road), Green (Safe Grass), Red (Danger/Debris).
2. The Canvas: A pixelated "Drone View" of a disaster zone.
3. Goal: Color-code the image so the "Rescue Rover" knows where to drive.

Game Structure & Level Logic (Teaching Segmentation):

- Level 1 [Road vs Water]: Instruction: "Teach the AI! Tap the Road pixels to paint them Grey. Tap Water to paint Blue."
- Level 2 [Debris Blocking]: There is rubble on the road. Paint it Red (Danger) so the Rover drives around it.

- Level 3 [Pathfinding]: You have segmented the image. Now draw a path that stays strictly on the "Grey" pixels.
- Level 4 [Power Lines]: Thin black lines are hard to see. Zoom in and segment them as "Danger" (Red).
- Level 5 [Auto-Segment]: Click "Run AI". It colors 80% of the map. You must manually fix the 20% errors (e.g., it thinks a puddle is a road).

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify grid coordinate systems.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Logic: The Rover movement function must check the color of the pixel before moving.
- Debugger: "🔧" instantly fills the segmentation map correctly.

P5

Act as an expert frontend game developer and logic instructor. Write a complete game called "Smart Alert Tuner" designed for 10-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. Live Feed: A simulated camera feed showing cars at an intersection.
2. Sensitivity Slider: A range from "Low Sensitivity" to "High Sensitivity".
3. Alert Log: A list of triggered alarms (Accident! Tailgating!).

Game Structure & Level Logic (Teaching False Positives/Negatives):

- Level 1 [The Miss]: Sensitivity is too low (0%). A crash happens, but no alarm goes off. Fail.
Instruction: "Increase Sensitivity!"
- Level 2 [The Annoyance]: Sensitivity is too high (100%). A bird flies by -> ALARM! A leaf falls -> ALARM! The operator gets angry.
- Level 3 [The Sweet Spot]: Find the setting (approx 60%) that catches the crash but ignores the bird.
- Level 4 [Subway Gate]: Detect "Tailgating" (Two people, one ticket). If you set it too high, it stops a mom holding a baby. Balance it.

- Level 5 [Night Mode]: At night, the camera is grainy (noisy). You must lower sensitivity to stop false alarms, but keep it high enough to catch real danger.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify threshold logic constants.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: `Trigger = (SignalStrength > SensitivityThreshold)`.
- Debugger: " " shows the numeric signal strength of passing objects.

P6

Act as an expert frontend developer and data science instructor. Write a complete game called "City Vector Tracker" designed for 11-12 year old students.

The game must be fully playable and functional as a standalone HTML5 application. Use a multi-camera security dashboard.

Strict Tablet & Mathematical Rules:

1. Multi-Cam Grid: 4 Camera feeds showing different parts of the city.
2. The Target: A "Missing Person" profile. NOT a face, but a "Feature Vector": { Shirt: Red, Hat: Blue, Bag: Yes }.
3. Privacy Mode: All faces are blurred. Students must track using only vectors (clothing/attributes).

Game Structure & Level Logic (Teaching Re-ID):

- Level 1 [Vector Matching]: Target has a Red Shirt. Tap the person in Cam 1 matching that vector.
- Level 2 [Cross-Camera]: Person leaves Cam 1 (Park) and enters Cam 2 (Subway). Re-identify them based on the vector.
- Level 3 [Occlusion]: Target is behind a crowd. Look for the specific "Blue Hat" attribute peaking out.
- Level 4 [Attribute Changes]: Target took off their jacket. The "Shirt Color" vector changed, but "Pants" and "Shoes" match. Update the tracking.
- Level 5 [The Network]: Track the target across 6 cameras over 1 minute. Draw their path on the map based on vector timestamps.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify attribute matching logic.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Logic: ``MatchScore = (Matches / TotalAttributes) * 100``.
- Debugger: "🔧" highlights the target in all active cameras.

Lesson 12

P1

Act as an expert frontend game developer and early-years UX designer. Write a complete game called "Traffic Tagger Bot" designed for 6-year-old students.

The game must be fully playable and functional as a standalone HTML5 application. Do not use external assets. Use system emojis (🚗, 🚌, 🚶, 🚦) and a street view canvas.

Strict Tablet & Interface Constraints:

1. The Stream: Vehicles move across the screen.
2. Tagging UI: Three big buttons at the bottom: "Car", "Bus", "Walker".
3. Action: Tap the moving object, then tap the correct button to "Teach" the AI what it is.

Game Structure & Level Logic (Teaching Data Labeling):

- Level 1 [Cars Only]: Only cars appear. Tap them and hit "Car". Instruction: "Teach the AI to see Cars!"
- Level 2 [The Bus]: Buses appear (bigger, yellow). They have priority. Tag them as "Bus".
- Level 3 [Walkers]: Pedestrians appear on the crosswalk. Tag them as "Walker".
- Level 4 [Priority Rules]: Tagging a Walker turns the light Red (Safety). Tagging a Car keeps it Green.
- Level 5 [Speed Round]: Traffic is fast. Tag 10 items correctly before time runs out to train the Model.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify collision and click detection logic.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Visuals: Draw a "Bounding Box" around the tagged entity.
- Debugger: "🔧" auto-tags all visible entities.

P2

Act as an expert frontend game developer and educational designer. Write a complete game called "Ambulance Swarm Clearer" designed for 7-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. Top-Down View: A grid of city streets with traffic cars (blue) and one Ambulance (red/flashing).
2. The Future Path: A ghost-line shows where the ambulance *will* go in 5 seconds.
3. Interaction: Tap blue cars to "Ping" them. They move out of the way (V2X Signal).

Game Structure & Level Logic (Teaching Predictive Control):

- Level 1 [The Path]: The Ambulance path is straight. Ping the 3 cars blocking it.
- Level 2 [The Turn]: The ambulance is turning left. Clear the intersection *before* it gets there.
- Level 3 [Gridlock]: Cars are stuck. You must ping them in the right order to make space (sliding puzzle).
- Level 4 [Timing]: The Ambulance is fast. You have 3 seconds to clear the route based on the "Future Line".
- Level 5 [The Swarm]: One tap on a lead car sends a signal to all cars behind it to move simultaneously.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify pathfinding and collision logic.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.

5. REFACTOR: Automatically correct any failed validation metrics.
[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Logic: `If (CarPosition == AmbulanceFuturePath) { Collision = True }`.
- Debugger: "🔧" clears all civilian traffic instantly.

P3

Act as an expert frontend game developer and logic instructor. Write a complete game called "Loop Sensor Master" designed for 8-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. Subterranean View: Show the road surface and the "Inductive Loops" (Sensors) buried underneath.
2. Data UI: Dynamic Bar Graphs rising as cars sit on top of the sensors.
3. Action: A "Flush Signal" button.

Game Structure & Level Logic (Teaching Sensor Logic):

- Level 1 [The Wait]: One car sits on the loop. The bar fills slowly. When it hits Red, hit "Flush" to turn the light Green.
- Level 2 [Capacity]: 5 cars are waiting. The bar fills super fast. Flush it before it overflows (Gridlock).
- Level 3 [False Alarm]: A bike sits on the sensor (Low metal mass). The bar fills only 10%. Don't flush! Wait for a car.
- Level 4 [Multi-Lane]: 4 lanes. 4 Bar Graphs. Flush the lane with the highest bar (Longest Queue).
- Level 5 [AI Automation]: Set a "Trigger Level" slider. Set it to 80%. Watch the system auto-flush when the queue gets too long.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify queue growth rates.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Visuals: The sensor should glow when a car is on top of it.

- Debugger: "🔧" sets all queues to 99% full.

P4

Act as an expert frontend game developer and math instructor. Write a complete game called "Green Wave Commander" designed for 9-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. The Dashboard: A live line graph showing "Average Traffic Speed".
2. Normal State: The line stays high (40mph).
3. The Signature: A sudden drop to 0mph signifies a crash/disaster.
4. Action: "Green Wave" button (Unlocks all lights).

Game Structure & Level Logic (Teaching Anomaly Detection):

- Level 1 [The Drop]: Watch the speed graph. When it drops to 0, tap "Green Wave" to clear the jam.
- Level 2 [Noise vs Signal]: The speed dips slightly (Slow Driver). Don't panic. Only react to the "Crash Signature" (Vertical drop).
- Level 3 [Location Spotting]: 3 Sectors. Sector 2 drops to zero. Apply Green Wave ONLY to Sector 2.
- Level 4 [Latency]: The graph is delayed by 2 seconds. Predict the crash based on the trend.
- Level 5 [Auto-Trigger]: Set a rule: "IF Speed < 5mph for > 3 seconds, THEN Trigger Green Wave." Test your rule.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify graph rendering logic.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: Graph data must update every frame (`requestAnimationFrame`).
- Debugger: "🔧" triggers a simulated crash event.

P5

Act as an expert frontend game developer and logic instructor. Write a complete game called "Traffic Brain Trainer" designed for 10-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. Scenario Cards: "Rainy Night + School Bus" vs "Sunny Day + Single Car".
2. Decision UI: Sliders for "Priority Score" (0-100).
3. Feedback: "AI Learning Score". The AI watches you and tries to copy your logic.

Game Structure & Level Logic (Teaching Imitation Learning):

- Level 1 [Basic Rules]: School Bus vs Car. You give Bus 90 priority, Car 10. AI learns: "Bus = Important".
- Level 2 [Environmental Variables]: Rain makes braking hard. Increase "Yellow Light Time" slider. Train AI to be safer in rain.
- Level 3 [Conflict]: Ambulance vs Fire Truck. Who goes first? Make a choice to set the "Policy".
- Level 4 [The Test]: The AI plays a scenario. It gets it wrong. Correct it manually to refine the weights.
- Level 5 [Batch Training]: 5 rapid scenarios. Make decisions fast. If your logic is consistent, the AI reaches 100% accuracy.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify scoring weights and consistency checks.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Logic: `AiAccuracy = 100 - Math.abs(UserChoice - AiPrediction)`.
- Debugger: "🔧" sets AI accuracy to 100%.

P6

Act as an expert frontend developer and data science instructor. Write a complete game called "Stadium Exodus Master" designed for 11-12 year old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Mathematical Rules:

1. The Map: A Stadium in the center, surrounded by a city grid.
2. The Agents: 1000 dots (Autonomous Vehicles) trying to leave.
3. Reward Dials: Three rotatable dials: "Speed Reward", "Safety Reward", "Eco Reward".

Game Structure & Level Logic (Teaching Reinforcement Learning):

- Level 1 [Maximize Speed]: Tune the AI to care ONLY about Speed. Cars crash, but they move fast. (High Speed Score, Low Safety Score).
- Level 2 [Safety First]: Dial up Safety. Cars move very slow, stopping at every corner. No crashes, but Gridlock ensues.
- Level 3 [Eco-Driving]: Dial up Eco. Cars accelerate smoothly (no gas guzzling). Flow improves.
- Level 4 [The Balance]: The Stadium lets out. You need High Speed + High Safety. Find the mathematical sweet spot on the dials.
- Level 5 [Adaptive Response]: Weather changes to "Ice". You must dynamically lower Speed Reward and boost Safety Reward in real-time to keep the flow moving without crashes.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors.
2. LOGIC CHECK: Trace data flow to confirm win conditions are reachable.
3. NUMERICAL AUDIT: Verify swarm movement physics.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{GlobalScore} = (\text{Speed} * W1) + (\text{Safety} * W2) + (\text{Eco} * W3)$.
- Debugger: "🔧" finds the optimal dial positions.

Lesson 13

P1

Act as an expert frontend game developer and educational designer. Write a complete game called "Shockwave Brake Sync" designed for 6-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. The View: A straight, horizontal single-lane highway with 15 colorful cars driving from left to right.
2. The Problem: A "Phantom Traffic Jam" appears as a red glowing wave moving backwards through the line of cars because the front car tapped its brakes.
3. Interaction: Native touch. Tap trailing cars to activate their "Smart Dash Warning" (turns the car green), which makes them slow down early and smoothly.

Game Structure & Level Logic (Teaching Predictive Braking Coordination):

- Level 1 [Catch the Wave]: Instruction: "A ghost traffic jam is moving backward! Tap 3 cars early to turn them green and stop the wave before it hits the back."
- Level 2 [The Speed Up]: The cars move faster. The red shockwave moves backward twice as fast. Tap 5 cars early to synchronize their brakes.
- Level 3 [Double Wave]: Two separate braking waves start at the front. Tap cars in both spots to smooth out the flow.
- Level 4 [The Long Chain]: The highway expands to 25 cars. Predict where the wave will go and tap 8 cars in a row to protect the tail end.
- Level 5 [Perfect Flow]: Stop a massive shockwave perfectly. Win condition requires zero cars coming to a complete, hard stop.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (6 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: `BrakingDistance = BaseDistance + (Speed * ReactionMultiplier)`. Green cars reduce `ReactionMultiplier` to 0.
- Debugger: " " freezes the moving shockwave to inspect vehicle distances.

P2

Act as an expert frontend game developer and educational designer. Write a complete game called "Highway Hero Route" designed for 7-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. The View: A two-lane highway. A slow tractor causes a backward-moving traffic wave in the right lane. An emergency fire truck is trying to enter from a left-side ramp.
2. Safety Zone: A blinking "Clearance Box" ahead of the fire truck.
3. Interaction: Tap vehicles inside the traffic wave to make them shift lanes or accelerate, clearing a 5-car gap for the fire truck.

Game Structure & Level Logic (Teaching Wave Dissipation for Emergency Vehicles):

- Level 1 [Clear the Ramp]: Instruction: "A slow truck made a traffic wave! Tap 3 cars to move them out of the lane so the Fire Truck can get on the highway."
- Level 2 [The Speed Merge]: The fire truck moves faster. You must clear a larger gap (5 cars long) before the fire truck reaches the highway entrance.
- Level 3 [Heavy Traffic]: More cars are stuck in the backward wave. Tap cars to move them left and right to distribute the density evenly.
- Level 4 [The Escort]: The fire truck enters. You must continuously clear the space 50 pixels directly ahead of it as it drives down the road.
- Level 5 [Gridlock Rescue]: A massive wave stops all traffic. Clear a perfect path for the fire truck using only 10 taps total.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (7 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.

- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: ``WaveSpeed = TrafficDensity * -2.5``.
- Debugger: " " clears all cars instantly to reset the highway flow.

P3

Act as an expert frontend game developer and educational designer. Write a complete game called "Forecast Flow Master" designed for 8-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. The View: A 4-way intersection controlled by a traffic light.
2. AI Predictive Feature: Translucent, glowing colored bands ("AI Forecast Paths") stretch out from incoming roads. The wider the band, the larger the incoming traffic wave will be in 5 seconds.
3. Interaction: Tap the traffic light to add or subtract seconds (+/- buttons) to the green light timer.

Game Structure & Level Logic (Teaching Predictive Signal Adjustments):

- Level 1 [Read the Future]: Instruction: "Look at the glowing AI Forecast Path! Extend the green light by 5 seconds to let the big traffic wave pass through before it bunches up."
- Level 2 [The Cross-Wave]: Two forecast paths show waves coming from the North and East at different times. Balance the timers to absorb both waves without stops.
- Level 3 [The Phantom Surge]: A sudden, thick forecast path appears from the West. You must quickly drain the intersection's current cars to make room for the surge.
- Level 4 [The Green Corridor]: Coordinate two lights in a row. Use the forecast paths to time the green lights so cars pass through both without hitting a red light.

- Level 5 [The Perfect Wave Absorber]: A massive stadium crowd releases traffic. Maintain a Global Flow Score above 90% by constantly adjusting light durations based on changing forecasts.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (8 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{PredictedDensity} = \text{CurrentVehicles} + (\text{ForecastGrowthRate} * \text{TimeDelta})$.
- Debugger: "🔧" highlights the dense cluster of cars that caused a jam.

P4

Act as an expert frontend game developer and educational designer. Write a complete game called "Predictive Barrier Control" designed for 9-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. The View: A multi-lane highway split by movable, dynamic lane barriers.
2. Dashboard Panel: A live line graph showing "10-Second Predictive Vehicle Density" for Lane A and Lane B. If a line goes above the red threshold, a shockwave is imminent.

3. Interaction: Drag a slider to shift the lane barriers left or right, instantly reallocating lanes (e.g., changing from 2-and-2 lanes to 3-and-1 lanes).

Game Structure & Level Logic (Teaching Predictive Resource Allocation):

- Level 1 [Read the Graph]: Instruction: "The graph predicts a massive spike in Lane A in 10 seconds. Move the barrier to give Lane A an extra lane to divide the shockwave!"
- Level 2 [The Tug of War]: Both lanes show alternating spikes on the graph. Shift the barrier back and forth ahead of time to match the graph's predictions.
- Level 3 [The Bottleneck]: The highway narrows. The predictive graph shows a sudden convergence wave. Reallocate lanes early to keep speed high.
- Level 4 [The Emergency Split]: An emergency vehicle is tracking down Lane B. The graph shows Lane B is overcrowding. Shift the barrier to clear out Lane B before the emergency vehicle arrives.
- Level 5 [Shockwave Deflection]: Manage a 3-lane system with two movable barriers. Use the predictive graph to prevent any lane's density from crossing into the red zone for 60 seconds.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (9 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{LaneCapacity} = \text{NumberOfLanes} * \text{BaseFlowRate}$. $\text{ShockwaveRisk} = \text{PredictiveDensity} / \text{LaneCapacity}$.
- Debugger: "🔧" shows optimal barrier positions as dotted lines on the highway.

P5

Act as an expert frontend game developer and educational designer. Write a complete game called "Echo Route Mitigator" designed for 10-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. The Map: A main highway bypass at the top, connected to three quiet, residential side streets below.
2. AI Dashboard: Shows "Flow Metrics" and "Echo Risk Indicators" (heat maps on side streets that fill up when vehicles get rerouted away from the highway).
3. Interaction: Tap intersections on side streets to change traffic light cycles and prevent the "Echo Effect" (secondary bottlenecks).

Game Structure & Level Logic (Teaching Secondary Feedback Loops in AI Routing):

- Level 1 [The First Echo]: Instruction: "A highway accident forces the AI to reroute cars into the neighborhood. Adjust the neighborhood traffic lights to be longer so this 'Echo' flow doesn't block local driveways."
- Level 2 [The Double Bounce]: Cars are rerouted through two residential streets at once. Balance both street lights to prevent a secondary backup at the local school zone.
- Level 3 [The Grid Ripple]: The highway clears up, but the neighborhood is still full. Use an AI dashboard slider to slowly transition traffic light patterns back to normal without creating new stops.
- Level 4 [The Delivery Surge]: A wave of delivery vans gets rerouted into a tight residential grid. Use short, fast light cycles to break the cluster into smaller, harmless groups of cars.
- Level 5 [Total Network Balance]: A major highway closure causes massive, multi-street echo effects. Maintain both high highway diversion speeds and clear neighborhood streets to win.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (10 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.

3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
 4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
 5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.
- [END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: ``EchoDensity = ReroutedVehicles * ResidentialConstrictionFactor``.
- Debugger: " " overlays arrows showing exactly where the rerouted traffic wave is overflowing.

P6

Act as an expert frontend game developer and data science instructor. Write a complete game called "Eulerian Flow Matrix" designed for 11-12 year old students.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Mathematical Rules:

1. The Map: A dense 6x6 macro city grid prone to cascading traffic shockwaves.
2. The Systems: Two toggles: "Eulerian Macro Control" (Fixed grid-based light timers) and "Micro-Agent Mesh" (Vehicles talking vehicle-to-vehicle to change speeds autonomously).
3. Reward Dashboard: Live telemetry displaying Average Velocity, Shockwave Dissipation Rate, and Emergency Path Latency.

Game Structure & Level Logic (Teaching Eulerian vs. Lagrangian Micro-Agent Optimization):

- Level 1 [The Manual Macro]: Try to manage a giant shockwave using only manual macro street-level optimization (changing column/row light patterns). Notice how hard it is to catch every wave.
- Level 2 [Deploy the Mesh]: Deploy the autonomous Micro-Agent Network. Watch the vehicle dots automatically adjust their inter-car distance to safely absorb the shockwaves without stopping.
- Level 3 [The Emergency Piercing]: A red emergency vehicle needs to cross the grid. The Micro-Agents must actively move aside to create a temporary "empty wave" path ahead of it.
- Level 4 [The Hybrid Sweet Spot]: A massive gridlock event occurs. Use macro-level Eulerian light corridors to handle bulk volume while activating the micro-agent mesh to prevent local street collapse.
- Level 5 [Dynamic Atmospheric Distortion]: Rain/Ice hits the city grid, changing traction values. Dynamically adjust the micro-agent safety margins via a slider to keep flow steady without crashes.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.

- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (11-12 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify swarm movement physics and grid cell data structures.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{GlobalEfficiency} = (\text{AverageVelocity} * 0.4) + (\text{DissipationRate} * 0.4) - (\text{EmergencyLatency} * 0.2)$.
- Debugger: "🔧" switches the canvas into a mathematical vector field view showing traffic pressures.

Lesson 14

P1

Act as an expert frontend game developer and educational designer. Write a complete game called "AI Shield Sync" designed for 6-year-old students. This game explicitly teaches the AI concept of Bounding Box Proximity Detection.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. The View: A clean radar screen with 5 colorful airplanes moving slowly across the canvas.

2. AI Learning Feature: Every plane has a glowing translucent circle around it called an "AI Safety Shield". This teaches students how AI visual trackers create mathematical boundary buffers (bounding boxes) to detect and prevent object collisions.
3. Interaction: Native touch. Tap an airplane to cycle its speed between "Slow", "Medium", and "Fast".

Game Structure & Level Logic (Teaching Bounding Box Proximity Detection):

- Level 1 [Shield Alert]: Instruction: "Two planes are getting too close! Tap a plane to change its speed so their glowing AI Safety Shields do not touch."
- Level 2 [The Crossroad]: Three planes are heading toward a central point. Adjust their speeds so they cross the intersection one by one without overlapping shields.
- Level 3 [Fast Jets]: Two fast-moving jets enter the radar. You must quickly slow down the trailing aircraft to build a safe buffer zone behind them.
- Level 4 [The Fleet]: Five planes are on the screen at the same time. Keep all AI Safety Shields blue. If two shields touch, they turn red and trigger a warning.
- Level 5 [Perfect Separation]: Keep the airspace clean for 45 seconds. Win condition requires zero shield overlaps and all planes exiting the screen safely.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (6 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. $\text{ShieldOverlap} = \text{Distance} < (\text{Radius1} + \text{Radius2})$.
- Debugger: "🔧" freezes all airplanes in place to inspect exact pixel distances between shields.

P2

Act as an expert frontend game developer and educational designer. Write a complete game called "Ghost Timeline Landers" designed for 7-year-old students. This game explicitly teaches the AI concept of Objective Functions & Cost Penalties.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. The View: A single runway lane with a horizontal "Ghost Timeline" running along the top showing 10 seconds of future arrivals.
2. AI Learning Feature: Translucent "Ghost Markers" show future states. A visible "Routing Waste Penalty Counter" explicitly teaches students how automation algorithms use negative point values (penalties) to discover the most optimal and efficient path.
3. Interaction: Tap an approaching plane to delay its arrival by 2 seconds, training the scheduling engine.

Game Structure & Level Logic (Teaching Objective Functions & Cost Penalties):

- Level 1 [Line Up]: Instruction: "Look at the Ghost Timeline! Tap the second plane to slide its marker back so it lands safely behind the first plane."
- Level 2 [The 3-Plane Gap]: Three planes are incoming. You must create a steady 3-second gap between each ghost marker on the timeline before they touch the runway.
- Level 3 [The Fast Lane]: A speedy jet is catching up to a slow propeller plane. Delay the propeller plane early or speed up the jet to optimize the arrival queue.
- Level 4 [Efficiency Penalty]: A counter tracks "Routing Waste". If you delay planes too much, you lose points. Find the exact timing to land 4 planes with zero penalties.
- Level 5 [The Landing Streak]: Land a sequence of 6 planes under tight traffic pressure without any overlapping timeline markers.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (7 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.

2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
 3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
 4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
 5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.
- [END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: ``LandingGap = NextPlaneArrivalTime - CurrentPlaneArrivalTime``. Penalty increases if ``LandingGap < 3`` or ``LandingGap > 8``.
- Debugger: " " auto-spaces the timeline to show the most efficient target arrival sequence.

P3

Act as an expert frontend game developer and educational designer. Write a complete game called "Predictive Ghost Trails" designed for 8-year-old students. This game explicitly teaches the AI concept of Predictive Motion Modeling & Vector Trajectory Overlap.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. The View: An open sky map where commercial jets travel.
2. AI Learning Feature: Curving, translucent paths ("Ghost Trails") project 5 seconds into the future based on physical momentum vectors. This explicitly teaches how predictive AI models calculate multiple future states simultaneously before committing to a decision.
3. Interaction: Drag an arrow on a plane to adjust its target turn angle. The Ghost Trail updates in real-time.

Game Structure & Level Logic (Teaching Predictive Motion Modeling):

- Level 1 [Predict the Arc]: Instruction: "Planes take time to turn! Look at the glowing Ghost Trails. Turn the blue plane so its trail does not cross the red plane's trail."
- Level 2 [The S-Curve]: Direct a large passenger jet through a narrow canyon of clouds. Adjust its turn angle early so its momentum trail does not hit the cloud walls.
- Level 3 [Intersection Alert]: Two trails show a future collision happening off-screen in 4 seconds. Change the turning radius of one jet to clear the intersection.
- Level 4 [The Heavy Jet]: Introduce a massive cargo plane with high momentum (turns very slowly). Route two nimble jets around its wide, sweeping Ghost Trail.
- Level 5 [Sky Grid Harmony]: Manage 4 large jets at once. Keep all their predictive trails completely clear of intersections for 60 seconds.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (8 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{TurningRadius} = (\text{Velocity}^2) / (\text{BaseGravity} * \tan(\text{BankAngle}))$. Ghost trail curvature updates based on `TurningRadius`.
- Debugger: "🔧" toggles a visual intersection point marker where two trails are predicted to clash.

P4

Act as an expert frontend game developer and educational designer. Write a complete game called "Autopilot Vector Align" designed for 9-year-old students. This game explicitly teaches the AI concept of Multi-Objective Optimization & Weight Pareto Fronts.

The game must be fully playable and functional as a standalone HTML5 application.

Strict Tablet & Interface Constraints:

1. The View: An airspace grid overlaid with moving wind vector arrows (jet streams).
2. AI Learning Feature: A tuning panel with three weighted sliders: "Fuel Efficiency", "Collision Avoidance", and "Size-Comfort". This teaches students how AI models balance competing goals where maximizing one constraint might degrade another.
3. Interaction: Adjust the sliders to reprogram the autopilot fleet's global cost function weights in real-time.

Game Structure & Level Logic (Teaching Multi-Objective Optimization):

- Level 1 [Headwind Drift]: Instruction: "A strong wind is pushing planes off course. Crank up the 'Collision Avoidance' slider so the AI recalculates tighter paths around each other."
- Level 2 [The Fuel Trap]: Planes are running low on fuel while fighting a crosswind. Boost 'Fuel Efficiency' to let the AI ride the wind vectors, but keep enough 'Collision Avoidance' to prevent crashes.
- Level 3 [Turbulence Shielding]: A bumpy wind zone appears. Large planes can handle it, but small planes crash. Balance 'Size-Comfort' and 'Fuel Efficiency' so small planes are routed completely around the storm.
- Level 4 [The Wind Tunnel]: A narrow jet stream speeds up planes heading East but stops planes heading West. Tune the AI sliders to split the fleet into separate, wind-optimized altitude paths.
- Level 5 [The Optimal Balance]: A sudden weather shift flips all wind vectors. Dynamically adjust your three sliders to maintain a Total Flight Score above 85 points.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (9 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{EffectiveVelocity} = \text{AircraftVelocity} + \text{WindVector}$. $\text{AI_PathCorrection} = (\text{CollisionWeight} * 2) - (\text{FuelWeight} * 0.5)$.
- Debugger: "🔧" shows the raw force lines of the wind vectors pushing against the aircraft hulls.

P5

Act as an expert frontend game developer and educational designer. Write a complete game called "Radar Cluster Filter" designed for 10-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students how AI algorithms separate meaningful patterns from background noise using Classification Filters. Students will learn the trade-off between "True Positives" (correctly finding a danger) and "False Positives" (triggering an alarm for no reason) in machine learning.

Strict Tablet & Interface Constraints:

1. The View: A crowded radar canvas filled with 40 blinking blips (commercial planes, tiny drones, weather noise, and migrating birds).
2. AI Filter Controls: Two interactive, rotatable dials labeled "Proximity Threshold" (how close dots must be to group them) and "Velocity Variance" (filtering out objects based on speed differences).
3. The AI Status Panel: A live dashboard tracking "True Positives" (real danger clusters caught) and "False Positives" (harmless objects accidentally flagged).
4. Interaction: Native touch. Turn the dials to update the AI's clustering logic in real-time. Legitimate danger clusters (planes on a collision course) glow red when detected.

Game Structure & Level Logic (Teaching Signal Detection & AI Threshold Tuning):

- Level 1 [Isolate the Threat]: Instruction: "There is too much background noise on the screen. Turn the Proximity dial up so the AI groups close dots together. Find the 1 real danger cluster!"
- Level 2 [The False Alarm Trap]: A flock of birds is flying closely together, tricking the AI into a false alarm. Adjust the Velocity Variance dial so the AI ignores slow-moving objects and filters out the birds.
- Level 3 [Drone Swarm Glitch]: Tiny hobby drones are filling the sky. Tune both dials to train the AI to ignore individual drone signals, but instantly flash an alert if a fast passenger jet enters their space.
- Level 4 [The Thundercloud Noise]: Weather static creates fake radar blips. Adjust the filters to help the AI see through the cloud static while keeping a 100% detection rate on the 3 real airplanes hidden inside.
- Level 5 [The Perfect Classifier]: Air traffic doubles. Turn both dials to find the mathematical sweet spot where the AI catches 100% of real threats (True Positives) while keeping False Alarms below 5%.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (10 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.

- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: `IsCluster = (Distance < ProximityThreshold) && (abs(Speed1 - Speed2) < VelocityVariance)`.`
- Debugger: " " toggles a raw text data grid showing the AI's confusion matrix (True Positives, False Positives, True Negatives, False Negatives).

P6

Act as an expert frontend game developer and data science instructor. Write a complete game called "4D Trajectory Matrix" designed for 11-12 year old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students how advanced AI models handle Multi-Objective Optimization across four dimensions (Space + Time). Students learn how an AI evaluates predictions to find the most efficient path while avoiding complex, shifting environmental constraints.

Strict Tablet & Mathematical Rules:

1. The Map: A large 3D airspace map projected onto a tablet-friendly 2D grid, featuring a massive, expanding weather storm in the center.
2. The System: An interactive "4D Trajectory Engine" panel. Tapping a plane opens four sliders to modify its path: X (Latitude), Y (Longitude), Z (Altitude Level), and T (Time/Speed synchronization).
3. Telemetry Dashboard: Displays real-time calculations from the AI engine: "Conflict Probability (%)", "Fuel Burn Rate", and "Storm Turbulence Risk".

Game Structure & Level Logic (Teaching 4D Trajectory-Based AI Scheduling):

- Level 1 [The Third Dimension]: Instruction: "A massive storm blocks the highway in the sky! Instead of wasting fuel turning around, boost the Altitude (Z) slider to command the AI to fly safely over the weather system."
- Level 2 [Time Synchronization]: Two jets are on a collision course at the exact same location. Modify the Time/Speed (T) profile of the second jet to delay its arrival at the intersection point by exactly 3 seconds.
- Level 3 [The Shifting Vector]: The storm cell is actively growing and changing shape over time. Plot a multi-point path that curves right around the expanding edge while keeping the Fuel Burn Rate low.
- Level 4 [The Convergence Slot]: Four planes arriving from different directions must pass through one narrow cloud gap. Coordinate their X, Y, Z, and T sliders so the AI slots them through the gap perfectly spaced apart.
- Level 5 [Dynamic Atmospheric Crisis]: The storm splits into two separate cells and unpredictable headwinds pick up. Dynamically adjust the 4D profiles of 6 planes at once to keep Conflict Probability at 0% while minimizing extra fuel burn.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (11-12 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify 4D vector tracking data structures and trajectory prediction math.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: `TrajectoryConflict = (abs(x2-x1) < 25) && (abs(y2-y1) < 25) && (abs(z2-z1) < 1) && (abs(t2-t1) < 4)`.
- Debugger: " " switches the radar into a side-profile altitude grid view showing flight paths relative to the storm heights.

Lesson 15

P1

Act as an expert frontend game developer and educational designer. Write a complete game called "Park Prompt Designer" designed for 6-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students the core concept of AI Compute Cost and Resource Trade-offs. Students learn that higher-quality AI generations require more computational power ("Gold Tokens") than basic ones ("Bronze Tokens"), forcing them to manage a limited digital budget.

Strict Tablet & Interface Constraints:

1. The View: A split screen. Left side has a canvas showing a 3x3 park grid. Right side has a "Token Bank" and generation buttons.
2. The System: The user spends tokens to generate park items (trees, swings, fountains).
 - Bronze Tokens (Cost: 1): Generates a simple, flat cartoon drawing.
 - Gold Tokens (Cost: 3): Generates a beautiful, animated, high-fidelity 3D-style item.
3. Interaction: Native touch. Tap a grid square, then select either the Bronze or Gold button to spend tokens and generate an item.

Game Structure & Level Logic (Teaching Compute Resource vs. Asset Quality):

- Level 1 [The Token Budget]: Instruction: "You have 10 tokens. Fill 4 empty squares in the park with high-quality Gold items before running out of power!"
- Level 2 [The Full Park Challenge]: You have 12 tokens. You must fill all 9 squares of the park grid. Balance your mix of cheap Bronze items and expensive Gold items to succeed.
- Level 3 [The Mayor's Request]: The Mayor wants at least three high-fidelity Gold items, but you only have 9 tokens total. Optimize your budget to fill 6 slots total.
- Level 4 [Fidelity Match]: A blueprint shows a target "Beauty Score." You have 15 tokens. Generate items to match or beat a score of 25 points. (Bronze = 1 point, Gold = 4 points).
- Level 5 [The Token Crunch]: You have only 8 tokens but must fill 8 slots while achieving a minimum Beauty Score of 12. Find the exact mathematical combination.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.

- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (6 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{RemainingTokens} = \text{TotalTokens} - (\text{BronzeCount} * 1) - (\text{GoldCount} * 3)$. $\text{BeautyScore} = (\text{BronzeCount} * 1) + (\text{GoldCount} * 4)$.
- Debugger: "🔧" refunds all spent tokens instantly to allow rapid testing of different design combinations.

P2

Act as an expert frontend game developer and educational designer. Write a complete game called "City Server Surge" designed for 7-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students about AI Server Infrastructure and Dynamic Resource Throttling. Students learn how a surge in user traffic forces an AI system to adjust token costs (Surge Pricing) to discourage non-urgent queries and protect hardware from overheating.

Strict Tablet & Interface Constraints:

1. The View: A large digital thermometer representing "Server Temperature" next to a conveyor belt of incoming citizen inquiries (represented as question bubbles: chat, maps, gaming).

2. The System: A rotatable slider dial labeled "Surge Cost Multiplier" (1x to 5x). High multiplier numbers stop citizens from spamming the system, lowering server load.
3. Interaction: Native touch. Drag the slider dial left or right to change token pricing in real-time as traffic waves grow or shrink.

Game Structure & Level Logic (Teaching Load Balancing & Thermal Management):

- Level 1 [Cool the Core]: Instruction: "Too many citizens are asking the AI questions! Drag the dial to 3x Surge Pricing to slow down incoming requests and drop the temperature back to green."
- Level 2 [The Peak Hour Wave]: A massive wave of traffic approaches. Turn the dial up early to block requests, then lower it carefully as traffic thins to keep citizens happy.
- Level 3 [Urgent vs. Casual]: Introduce blue "Emergency Map Requests" that ignore pricing and always pass through, alongside grey "Casual Chat Requests." Keep the core from overheating while processing all blue requests.
- Level 4 [The Balancing Act]: Maintain a "Citizen Satisfaction Meter" above 70% while keeping the Server Temperature under 90°C during a variable traffic loop. High surge prices lower satisfaction but save the server.
- Level 5 [Total Grid Lockdown]: A massive, chaotic traffic spike happens. Dynamically adjust the pricing dial between 1x and 5x to survive 45 seconds without triggering a system crash or losing customer satisfaction.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (7 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{IncomingTrafficRate} = \text{BaseTraffic} / \text{SurgeMultiplier}$. $\text{TemperatureChange} = (\text{IncomingTrafficRate} * 2.5) - \text{CoolingRate}$.
- Debugger: "🔧" vents the system heat, instantly dropping the server temperature back to zero.

P3

Act as an expert frontend game developer and educational designer. Write a complete game called "Passport Budget Allocator" designed for 8-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students the concept of Token Budgets across Multi-Modal AI models. Students learn that different data types (Text, Databases, Images) require vastly different amounts of token resources, and must prioritize requests to keep an application running efficiently.

Strict Tablet & Interface Constraints:

1. The View: A digital passport verification dashboard. Citizens wait in line to process their passports. Each citizen requires a mix of three AI tasks.
2. The Math Matrix:
 - Text Reading (Name check) = Costs 2 Tokens.
 - Application Lookups (Database query) = Costs 5 Tokens.
 - Image Face Matching (Computer vision scan) = Costs 10 Tokens.
3. Interaction: A fixed daily budget of 100 Tokens. Tapping a citizen reveals checkboxes to turn their required verification tasks on or off.

Game Structure & Level Logic (Teaching Priority Queue Optimization):

- Level 1 [The Daily Cap]: Instruction: "You have a 100-token budget. Process 5 basic citizens who only require Text Reading (2 tokens) and Application Lookups (5 tokens)."
- Level 2 [The Security Surge]: A group of VIP citizens arrives. They **must** have Image Face Matching (10 tokens) processed. Sacrifice regular citizen lookups to ensure the VIPs are fully scanned within the 100-token budget.
- Level 3 [Maximum Throughput]: Pass 10 citizens through the line. You must budget perfectly so that you do not run out of tokens before the 10th citizen is processed.
- Level 4 [The Backup Crisis]: The citizen queue is getting long. Reject non-essential image scans for standard citizens to process double the amount of people using text and application steps only.
- Level 5 [The Optimal Passport Day]: Process a mixed line of 12 complex citizens. Achieve a 100% processing rate for critical items while spending exactly 100 tokens or less without leaving anyone stuck.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.

- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (8 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{TokensSpent} = (\text{TextCount} * 2) + (\text{LookupCount} * 5) + (\text{ImageCount} * 10)$. Loss condition triggers if $\text{TokensSpent} > 100$ before objectives are met.
- Debugger: "🔧" adds an extra emergency 20 tokens to the daily bank for debugging testing cycles.

P4

Act as an expert frontend game developer and educational designer. Write a complete game called "Model Training Tycoon" designed for 9-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students about AI Model Training Budgets and the concept of "Early Stopping." Students learn that training an AI model drains computational resources over time, and they must calculate when to halt training before hitting diminishing returns or depleting their token pools.

Strict Tablet & Interface Constraints:

1. The View: Three live loading graphs representing the training cycles of an Audio AI, Text AI, and Image AI.

2. The Dashboard: A shared master pool of 100 Compute Tokens. Accuracies climb as training runs, but token burn accelerates.
3. Interaction: Native touch. Tap a prominent "HALT TRAINING" button above any of the three models to lock in its current accuracy and freeze its token drain.

Game Structure & Level Logic (Teaching Diminishing Returns & Training Halts):

- Level 1 [First Halting]: Instruction: "Start training the Text AI. Watch its accuracy curve flatten. Hit HALT when it reaches 80% accuracy before it burns all 100 tokens!"
- Level 2 [Triple Allocation]: Train all three models together. You have 100 tokens total. You must halt each model individually so that all three reach at least 75% accuracy before tokens hit 0.
- Level 3 [The Image Crunch]: Image AI models burn tokens twice as fast (2 tokens per second) compared to Audio and Text. Balance your training timeline so the Image AI gets enough juice without starving the other two.
- Level 4 [Diminishing Returns Challenge]: A graph shows that after 85% accuracy, training speed slows down by 4x. Strategically halt training right at 85% to save tokens for a secondary model that needs to reach its threshold.
- Level 5 [The Master Fleet Optimizer]: Achieve an average model accuracy of 88% across all three systems using exactly 100 tokens. Requires precise timing and utilization of early stopping mechanics.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (9 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{Accuracy} = \text{BaseLog}(\text{TimeElapsed})$. $\text{TokenBurnRate} = \text{AudioRate}(1) + \text{TextRate}(1) + \text{ImageRate}(2)$.

- Debugger: "🔧" slows down the simulation speed by 0.5x to observe token burn inflection points.

P5

Act as an expert frontend game developer and educational designer. Write a complete game called "Gov AI Engine Broker" designed for 10-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students how to evaluate and deploy large-scale AI Models based on trade-offs between Speed, Cost (Tokens), and Accuracy. Students will understand Pareto Efficiency by selecting the optimal engine configuration for varying municipal workloads.

Strict Tablet & Interface Constraints:

1. The View: A procurement catalog displaying 3 distinct AI Engines:
 - "TinyLLM" (Fast, Cheap, Low Accuracy)
 - "MidCloud AI" (Balanced Speed/Cost, Moderate Accuracy)
 - "Maximus Ultra" (Slow, Highly Expensive Token Cost, Near-Perfect Accuracy)
2. The Problem: Incoming citizen service tickets with clear compliance conditions (e.g., "Must be 95% accurate" or "Must process within 2 seconds").
3. Interaction: Tap an incoming ticket batch, then tap one of the three AI Engines to route the workload.

Game Structure & Level Logic (Teaching Multi-Criteria Decision Analysis):

- Level 1 [Emergency Accuracy]: Instruction: "A batch of medical translation requests arrives. They demand 98% Accuracy! Route them to 'Maximus Ultra' despite the high token cost."
- Level 2 [The Tax Bot Spam]: Thousands of simple, routine address-change forms arrive. They must be handled quickly and cheaply. Route them to 'TinyLLM' to save your token bank.
- Level 3 [The Restricted Budget]: You are handed a tight budget of 500 tokens. You must process 3 separate batches of mixed data (Legal, Chat, Forms). Balance engine selections so you don't run out of tokens before completing the batches.
- Level 4 [The Time Crunch]: A traffic congestion emergency requires immediate response analysis (under 1.5 seconds per query). Route the requests to an engine that fulfills the time constraint without dropping below the baseline accuracy safety line.
- Level 5 [The Optimal Procurement Matrix]: Route 5 consecutive dynamic workload surges perfectly. You must achieve a Total Operational Efficiency Score of 90%+ without breaking your tokens, latency limits, or accuracy boundaries.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.

- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (10 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{EfficiencyScore} = (\text{AccuracyWeight} * \text{Acc}) + (\text{SpeedWeight} * \text{Spd}) - (\text{TokenCostWeight} * \text{Cost})$.
- Debugger: "🔧" shows the raw efficiency trade-off curve (Pareto Front) overlaying the engine catalog.

P6

Act as an expert frontend game developer and data science instructor. Write a complete game called "Cluster Compute Matrix" designed for 11-12 year old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students the physical infrastructure limitations of training Large Language Models and Neural Networks. Students learn how high-density matrix math workloads place severe physical stress on hardware, and must balance training step speed against core chip temperatures and power grids.

Strict Tablet & Mathematical Rules:

1. The Map: A telemetry dashboard displaying a cluster of 4 high-performance AI Server Nodes (Node Alpha, Beta, Gamma, Delta).

2. Live Metrics: Each node tracks: "FLOPs Utilization (%)", "Core Temperature (°C)", and "Token Processing Throughput (Tokens/sec)".
3. Interaction: Sliders for each node to allocate "Compute Power %" and a toggle button to active liquid nitrogen cooling bursts.

Game Structure & Level Logic (Teaching Infrastructure Scaling and Thermal Thresholds):

- Level 1 [Thermal Runway]: Instruction: "Node Alpha is running a massive Transformer training loop. Its core temperature is spiking past 95°C! Slide its Compute allocation down to 50% to prevent a hardware crash."
- Level 2 [The Batch Balance]: A massive dataset arrives. Distribute the workload evenly across all 4 nodes (25% each) so the entire cluster trains the model at a steady pace without any single node overheating.
- Level 3 [Liquid Nitrogen Cooling Shifting]: A sudden model convergence spike forces Node Beta and Gamma to run hot. Coordinate your limited cooling bursts between the two nodes to keep them alive while keeping throughput high.
- Level 4 [The Node Failure Reroute]: Node Delta completely breaks down for maintenance. You must split its ongoing token processing workload among Alpha, Beta, and Gamma without overloading their power thresholds.
- Level 5 [The Hyper-Parameter Peak]: Run a massive deep learning cluster at maximum capacity for 60 seconds. Maintain an aggregate cluster processing speed of over 50,000 Tokens/sec while keeping all core temps strictly under the 100°C catastrophic failure line.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (11-12 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify server temperature equations and linear programming load distribution logic.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{HeatGeneration} = (\text{ComputePower}^{1.8}) * 0.05$. $\text{CoreTemp} = \text{CoreTemp} + \text{HeatGeneration} - (\text{CoolingActive} ? 8 : 2)$.

- Debugger: "🔧" switches the UI into an infrared thermal imaging view displaying heat dissipation vectors across the rack.

Lesson 16

P1

Act as an expert frontend game developer and educational designer. Write a complete game called "Emoji Sentiment Sorter" designed for 6-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students the concept of Data Labeling and Supervised Learning in AI Sentiment Analysis. Students learn that an AI does not automatically know if a word is happy or sad; it relies on humans to label training data to teach it the emotional weight of specific vocabulary.

Strict Tablet & Interface Constraints:

1. The View: A split screen. The top contains floating word bubbles with simple text (e.g., "Great", "Broken", "Happy", "Bad", "Love", "Mad"). The bottom contains two large bins: a Green "Happy Bin" (Positive) and a Red "Sad Bin" (Negative).
2. The System: Dragging a bubble into the correct bin adds a data point to the AI's memory bank and updates a "Smart Robot" face in the corner.
3. Interaction: Native touch drag-and-drop mechanics.

Game Structure & Level Logic (Teaching Vocabulary Weighting & Training Sets):

- Level 1 [The Happy Start]: Instruction: "Help the AI learn happy words! Drag 3 smiling words like 'Fun' and 'Good' into the Green Bin."
- Level 2 [The Sad Storm]: Disappointed citizen words are falling fast! Sort 5 negative vocabulary words like 'Hurt' and 'No' into the Red Bin to train the AI's negative memory.
- Level 3 [The Mixed Batch]: A fast-moving mix of 8 positive and negative words fall at once. Sort them into the correct bins before they hit the floor to keep the AI training accurate.
- Level 4 [The Score Counter]: Introduce a "Training Accuracy Meter." Every correct sort boosts accuracy. Reach 90% accuracy with 10 falling words to pass.

- Level 5 [The Ultimate Sorter]: Train the AI perfectly during a 30-second rush. Pass the level by achieving zero mislabeled words to unlock the AI's fully trained happy robot mode.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (6 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{AIAccuracy} = (\text{CorrectLabels} / \text{TotalProcessedLabels}) * 100$. Loss condition triggers if bubbles hit the floor or enter the wrong bin too many times.
- Debugger: "🔧" labels all floating words with explicit "+1" or "-1" markers for rapid testing.

P2

Act as an expert frontend game developer and educational designer. Write a complete game called "Lexicon Score Master" designed for 7-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students how Natural Language Processing (NLP) systems use a Lexicon (a mathematical rulebook dictionary) to calculate numerical sentiment scores. Students will learn how an AI sums up individual word values to decide if a whole sentence is Positive, Negative, or Neutral.

Strict Tablet & Interface Constraints:

1. The View: A central clipboard showing a citizen text card (e.g., "The park is very clean but the swings are broken"). The bottom displays a "Rulebook Dictionary" with numerical word weights (e.g., clean = +2, broken = -2, very = multiplier).
2. The System: Three interactive grading buttons: "Positive (+)", "Neutral (0)", and "Negative (-)" based on the math sum.
3. Interaction: Native touch. Tap the correct grading button to classify the sentence after reading the rulebook values.

Game Structure & Level Logic (Teaching Sentence Sentiment Aggregation):

- Level 1 [Simple Addition]: Instruction: "Look at the sentence: 'I love the new bus!' Check the dictionary: 'love' is +2. Tap the Positive (+) button!"
- Level 2 [The Zero Balance]: A sentence has mixed words: "The road is smooth (+2) but the traffic is bad (-2)." Calculate the total sum (0) and tap the Neutral button.
- Level 3 [The Word Multiplier]: Introduce the word "Very" or "Super" which doubles the score of the next word. Calculate a high-intensity message score to categorize it correctly.
- Level 4 [The Speed Grade]: Five citizen text cards slide onto the clipboard one after another. Grade them using the rulebook math within a 30-second timer.
- Level 5 [The Perfect Algorithm]: Grade a tricky stack of 6 sentences featuring conflicting keywords. Win by matching the exact mathematical sentiment calculations without a single scoring error.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (7 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{SentenceScore} = \text{Sum}(\text{WordWeights}) * \text{Multiplier}$. Positive if $\text{Score} > 0$, Negative if $\text{Score} < 0$, Neutral if $\text{Score} == 0$.
- Debugger: "🔧" shows the live mathematical equation overlaying the text card words.

P3

Act as an expert frontend game developer and educational designer. Write a complete game called "Civic Feedback Router" designed for 8-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students how advanced AI systems combine Sentiment Scores with Intent Classification to sort real-world text data. Students will learn how an AI identifies keywords to find the **topic** while reading the **emotion** to decide how fast to route complaints to city departments.

Strict Tablet & Interface Constraints:

1. The View: A conveyor belt of incoming citizen email cards. Each card displays an AI telemetry readout showing its computed Sentiment Score (-5 to +5) and Topic Tag.
2. The Targets: Three municipal chutes labeled "Parks & Recreation", "Transit Authority", and "Waste Management".
3. Interaction: Native touch. Drag the email card into the correct chute. Critical emergency complaints (highly negative scores) pulse red and must be sorted instantly.

Game Structure & Level Logic (Teaching Multi-Factor Routing Rules):

- Level 1 [Find the Department]: Instruction: "An email says 'The bus was late again!' Its score is negative. Drag it to the Transit Authority chute!"
- Level 2 [The Negative Priority]: Two emails appear. One is an angry message about a broken water fountain (-4 score), the other is a happy note about flowers (+2). Route the urgent negative message first to clear the alert.
- Level 3 [The Multi-Chute Rush]: Emails fill up the conveyor belt. Route 8 messages to Parks, Transit, and Waste based on their keywords before the belt overflows.
- Level 4 [The Tricky Filter]: Some emails mention two topics, but their sentiment score points to the primary issue (e.g., "I love parks, but the trash bins there are overflowing!"). Use the AI sentiment marker to send it to the right department.
- Level 5 [The Chaos Shift]: A massive storm hit the city. 12 urgent, high-intensity negative feedback cards flood the dashboard. Route them perfectly without misplacing a single card to pass.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.

- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (8 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{PriorityWeight} = \text{abs}(\text{SentimentScore}) * \text{UrgencyMultiplier}$. Cards with $\text{PriorityWeight} > 7$ trigger a red flash and faster belt speed.
- Debugger: "🔧" stops the conveyor belt entirely to examine the keyword extraction arrays.

P4

Act as an expert frontend game developer and educational designer. Write a complete game called "Chatbot Tone Tuner" designed for 9-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students how Generative AI models use Conditional Generation to adjust their output Persona and Tone. Students will learn that an AI chatbot modifies its text behavior based on the detected sentiment of the user to maximize conversational satisfaction and de-escalate frustration.

Strict Tablet & Interface Constraints:

1. The View: A chat window showing a live chat message from an angry or happy citizen. Below the window is a "Tone Selector Panel" with three distinct AI persona chips: "Enthusiastic Mode", "Robotic Mode", and "Apologetic Mode".

2. The System: Selecting a tone updates the AI's generation parameters, outputting a preview reply box.
3. Interaction: Native touch. Tap a persona chip, then tap "SEND REPLY" to match the citizen's emotional state.

Game Structure & Level Logic (Teaching Persona Tuning & Response Optimization):

- Level 1 [Match the Anger]: Instruction: "A citizen is furious that their trash wasn't picked up. Tap 'Apologetic Mode' to lower their frustration bar!"
- Level 2 [The Cheerleader]: A citizen sends a happy message thanking the city for a fun festival. Use 'Enthusiastic Mode' to match their high positive energy and score bonus points.
- Level 3 [The Bureaucratic Balance]: A citizen asks a dry, purely technical question about parking permit forms. Use 'Robotic Mode' to deliver factual data quickly without annoying them with fake emotions.
- Level 4 [The Pivot Challenge]: A citizen starts happy but changes mood mid-sentence ("I liked the library, but the new guard was mean"). Analyze the final text sentiment shift and pick the right tone to save the chat rating.
- Level 5 [The Master Help Desk]: Manage a rapid succession of 6 diverse citizen chats. Keep the average "Public Satisfaction Meter" above 85% by picking the perfect AI conversational persona for every query.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (9 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{SatisfactionDelta} = \text{ToneMatchMatrix}[\text{CitizenSentiment}][\text{SelectedTone}]$. Incorrect pairings reduce the meter.
- Debugger: " " displays the citizen's raw sentiment value (-10 to +10) directly next to their face.

P5

Act as an expert frontend game developer and educational designer. Write a complete game called "Civic Trend Cloud" designed for 10-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students how AI text mining tools use Term Frequency-Inverse Document Frequency (TF-IDF) and Clustering to generate live visual summaries (Word Clouds). Students will learn how AI detects emerging public issues via changes in word size (frequency) and color (sentiment shifts).

Strict Tablet & Interface Constraints:

1. The View: A dynamic, shuffling Word Cloud on the left side of the tablet screen. Words like "Pothole", "Park", "Buses", "Clean", and "Delays" float and change size.
2. The AI Telemetry Indicators: Word size represents frequency (how many people are talking about it). Word color represents sentiment (Bright Green = Praises, Blinking Red = Severe Complaints).
3. Interaction: Tap a growing, flashing red word to expand its underlying citizen report data, then tap a "Deploy Response Action" button to send city crews to fix it.

Game Structure & Level Logic (Teaching Real-Time Feature Identification):

- Level 1 [Spot the Spike]: Instruction: "The word 'Pothole' is growing larger and turning bright red! Tap it to see the complaints and deploy a repair truck before the town gets angry."
- Level 2 [The False Alarm Filter]: The word 'Snow' is huge because everyone is excited about winter, but its color is Green (happy). Ignore it and find the smaller, deep red word 'Slippery' to send out salt trucks instead.
- Level 3 [The Sneaky Crisis]: A tiny word 'Water' suddenly flashes dark red. It has low frequency but dangerous sentiment intensity. Catch it and investigate before it turns into a massive city-wide issue.
- Level 4 [The Double Cluster]: Two word clusters spike at once: 'Delays' in transit and 'Overcrowded' in parks. Prioritize your limited city budget to deploy responses to both clusters before satisfaction plummets.
- Level 5 [City Matrix Stability]: Monitor a highly chaotic, fast-moving civic word cloud for 60 seconds. Successfully catch and handle 5 hidden emerging negative clusters while letting positive words grow.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (10 years old).

- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{WordSize} = \text{BaseSize} + (\text{Frequency} * 2.2)$. $\text{ColorHue} = \text{NeutralGrey} + (\text{SentimentScore} * \text{ColorShiftStep})$.
- Debugger: "🔧" prints a live ranking list of all words sorted by mathematical frequency-weighting vectors.

P6

Act as an expert frontend game developer and data science instructor. Write a complete game called "Mayor's Sentiment Matrix" designed for 11-12 year old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students how policy makers use Macro-Level Sentiment Aggregation and Data Dashboards to make decisions. Students learn how an AI reads thousands of messy data streams, filters out statistical noise, and presents global metrics to help optimize public resources.

Strict Tablet & Mathematical Rules:

1. The Map: A birds-eye view of a digital city divided into four sectors: North, South, East, and West.
2. The Dashboard: A live line graph tracking the "Global Civic Approval Index (%)" alongside a bar chart displaying localized sector sentiment scores (-100 to +100).
3. Interaction: Sliders to allocate a shared city budget across three resources: "Infrastructure Fund", "Social Services", and "Green Space Upkeep".

Game Structure & Level Logic (Teaching Macro Optimization & Feedback Loops):

- Level 1 [The Sector Slump]: Instruction: "Look at the AI dashboard! The East Sector sentiment is plummeting due to road issues. Slide budget into the Infrastructure Fund and target the East to pull their score out of the red."
- Level 2 [The Echo Complaint]: Fixing roads in the East made the West angry about construction noise. Read the macro charts to re-balance funding into Social Services to stabilize the regional blowback.
- Level 3 [The Budget Crunch]: A city recession cuts your budget in half. Use the AI sentiment forecasting tool to find the absolute minimum spending required in each sector to prevent your Global Approval Index from dropping below 60%.
- Level 4 [The Sudden Event Surge]: A flash flood hits the North and South sectors simultaneously. Use real-time macro telemetry to pull funds from quiet zones and shield the affected populations before a catastrophic political collapse.
- Level 5 [The Election Run]: Maintain a Global Approval Index of 80% or higher for a full 60-second runtime while navigating fast-changing public opinions, seasonal demands, and unexpected infrastructure emergencies.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (11-12 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify aggregate dashboard tracking logic and linear system modeling formulas.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{GlobalApproval} = \text{Mean}(\text{SectorSentimentScores})$. $\text{SectorSentiment} = \text{SectorSentiment} + (\text{AllocatedBudget} * 0.4) - (\text{SectorNegativityGrowth} * 1.5)$.
- Debugger: "🔧" switches the dashboard into an algebraic equation sheet to view the exact rate-of-change metrics for each sector.

Lesson 17

P1

Act as an expert frontend game developer and educational designer. Write a complete game called "City Treasure Optimizer" designed for 6-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students the core AI concept of Cost-Benefit Optimization in automated resource allocation. Students learn that an AI system must balance a strict budget constraint while deciding whether to deploy low-cost basic solutions or high-cost advanced solutions to fix city problems.

Strict Tablet & Interface Constraints:

1. The View: A split screen. Left side shows a 3x3 city map grid with broken spots (potholes, dark streetlights, broken benches). Right side contains a "Coin Box" bank and two fix buttons.
2. The System: The user spends coins to fix city problems:
 - Bronze Fix (Cost: 1 Coin): A simple, fast patch that rewards +1 City Happiness.
 - Gold Tech Fix (Cost: 3 Coins): Installs an advanced AI smart sensor upgrade that rewards +4 City Happiness.
3. Interaction: Native touch. Tap a broken grid square, then tap either the Bronze or Gold button to spend coins.

Game Structure & Level Logic (Teaching Resource Optimization Constraints):

- Level 1 [The Coin Cap]: Instruction: "You have 10 coins. Use the Gold Tech Fix on 3 broken spots to see how smart sensors make people super happy before your coins run out!"
- Level 2 [The Whole City Fix]: You have 12 coins. You must fix all 9 broken squares on the grid. Balance your mix of cheap Bronze patches and expensive Gold upgrades to succeed.
- Level 3 [The Happiness Target]: The town demands a total City Happiness Score of 20, but you only have 11 coins. Optimize your budget combinations to hit the target.
- Level 4 [The Double Breakdown]: A storm breaks things twice as fast. You have 15 coins. Plan ahead and allocate funds to hit a target score of 25 points.
- Level 5 [The Perfect Audit Budget]: You have exactly 8 coins and must fix 8 slots while hitting a minimum Happiness Score of 14. Find the exact mathematical combination.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (6 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{RemainingCoins} = \text{TotalCoins} - (\text{BronzeCount} * 1) - (\text{GoldCount} * 3)$. $\text{HappinessScore} = (\text{BronzeCount} * 1) + (\text{GoldCount} * 4)$.
- Debugger: "🔧" refunds all spent coins instantly to test different allocation paths quickly.

P2

Act as an expert frontend game developer and educational designer. Write a complete game called "Audit Route Master" designed for 7-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students how AI systems use Triaging and Conditional Routing to minimize computational overhead. Students learn that routing simple data to complex models wastes government funds, and they must sort incoming tasks to match model complexity with document difficulty.

Strict Tablet & Interface Constraints:

1. The View: A conveyor belt of incoming financial aid applications (represented as document folders with page size icons). Below the belt are two AI Audit Models: "Linear Parser" (Low Compute Cost) and "Deep Matrix Auditor" (High Compute Cost).
2. The System: Document types:
 - Simple Forms (1 page, basic text): Best for Linear Parser (Cost: 1 token).
 - Complex Tax Portfolios (Multi-page, charts): Best for Deep Matrix Auditor (Cost: 5 tokens).
3. Interaction: Native touch. Drag the document folders into the correct AI model intake chute.

Game Structure & Level Logic (Teaching Efficient Task Distribution):

- Level 1 [The Right Machine]: Instruction: "A simple 1-page form arrives! Drag it to the 'Linear Parser' to process it cheaply and save government tokens."
- Level 2 [The Complex Portfolio]: A massive stack of multi-page tax documents arrives. Route them to the 'Deep Matrix Auditor' so the AI can read the complex charts, but monitor your token bank.
- Level 3 [The Token Crunch]: You are given a strict budget of 25 tokens. Process a mixed queue of 9 incoming documents without running out of compute resources.
- Level 4 [The Speed Surge]: The conveyor belt speeds up. Documents pile up quickly. Route 12 items to their optimal models under tight time constraints without making a routing mistake.
- Level 5 [Perfect Efficiency Score]: Process a chaotic sequence of 10 mixed folders. Achieve a Total Processing Efficiency of 100% by ensuring zero simple forms are wasted on the expensive deep model.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (7 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{TotalTokensSpent} = (\text{SimpleInLinear} * 1) + (\text{ComplexInDeep} * 5) + (\text{SimpleInDeep} * 5) + (\text{ComplexInLinear} * 10)$. (Note: Penalty cost applies for bad matching).
- Debugger: "🔧" clears the active conveyor belt to reset the triaging simulation.

P3

Act as an expert frontend game developer and educational designer. Write a complete game called "Compute Fee Meter" designed for 8-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students the mechanics of Dynamic Resource Pricing in cloud infrastructure. Students learn that processing large corporate files drains severe server resources (RAM and tokens), and they must adjust billing parameters to scale fees dynamically based on system consumption metrics.

Strict Tablet & Interface Constraints:

1. The View: A live data center dashboard showing corporate uploads sliding into an AI Document Parser. Next to it is a live "RAM Usage Bar" and a "System Load Thermometer."
2. The Controls: Two slider bars labeled "Page Count Multiplier" and "RAM Usage Premium Fee."
3. Interaction: Adjust the sliders to scale the financial billing rates in real-time as different sizes of corporate audit data are uploaded.

Game Structure & Level Logic (Teaching Infrastructure Cost Scaling):

- Level 1 [Scale for Size]: Instruction: "A mega corporation is uploading a huge 100-page file! Slide the 'Page Count Multiplier' up to charge them the correct amount of tokens for using so much server power."
- Level 2 [The RAM Overload]: Small but hyper-complex spreadsheet data files are uploaded, causing the RAM bar to spike into the red. Boost the 'RAM Usage Premium Fee' to offset infrastructure wear-and-tear.
- Level 3 [The System Crash Threat]: A wave of heavy corporate audits attacks the server simultaneously. Crank up both billing sliders to throttle non-urgent uploads and cool the system load back to normal.
- Level 4 [The Balanced Budget]: Maintain a "Corporate Fairness Rating" above 75% while collecting at least 500 tokens of compute revenue during a variable upload cycle. Overcharging small businesses lowers fairness.
- Level 5 [The Optimal Billing Matrix]: Survive a 45-second high-traffic corporate tax season rush. Dynamically adjust both sliders to maintain zero server crashes while matching exactly 100% of physical compute costs with billed tokens.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.

- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (8 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{BilledFee} = (\text{PageCount} * \text{PageMultiplier}) + (\text{RamUsage} * \text{RamPremium})$. $\text{SystemLoad} = \text{SystemLoad} + (\text{FileWeight} / (\text{BilledFee} + 1)) - \text{CoolingFactor}$.
- Debugger: "🔧" displays the raw cost-per-megabyte analytics numbers directly over incoming uploads.

P4

Act as an expert frontend game developer and educational designer. Write a complete game called "District Crisis Auditor" designed for 9-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students the concept of Active Learning and Semi-Automated Systems. Students learn how an AI model continuously processes and classifies incoming data streams while relying on human operators to execute manual overrides when unexpected infrastructure anomalies occur.

Strict Tablet & Interface Constraints:

1. The View: A live dashboard showing a list of city districts (District 1 to District 4), each with an "Infrastructure Health Bar." A stream of citizen emergency tickets cascades down the center.

2. The System: Each ticket requires active categorization ("Bridge", "Water", "Road"). Correct labeling helps the AI automate budget distribution. A prominent red "EMERGENCY OVERRIDE" button allows manual cash routing.

3. Interaction: Native touch. Tap categories to label tokens, and tap the override button to manually dump funds into a collapsing district.

Game Structure & Level Logic (Teaching Semi-Automated Allocation Systems):

- Level 1 [Continuous Labeling]: Instruction: "A flood of tickets is incoming. Label 5 tickets as 'Water' or 'Road' so the AI can automatically route repair funds to save District 1!"

- Level 2 [The AI Delay]: The ticket volume doubles, and the AI cannot process the math fast enough. District 3's health is plunging. Hit the 'EMERGENCY OVERRIDE' button to force a manual cash dump to save it.

- Level 3 [The Misclassification Storm]: Malicious spam tickets enter the queue, confusing the AI's data labels. Rapidly clean up the data labeling inputs manually to ensure valid budget streams reach the true broken zones.

- Level 4 [The Double Sector Crash]: Two districts face infrastructure collapse simultaneously. Balance your time between typing accurate AI labels for bulk funding and slamming manual overrides to patch local failures.

- Level 5 [Total Grid Optimization]: Keep all four district health bars above 50% for a 60-second operational window under intense, chaotic ticket traffic. Perfect synchronization of manual overrides and AI tracking is required.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.

- Target Hardware: Tablet devices.

- UI/UX: Native touch interactions, responsive layout, fluid scaling.

- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.

- Level Access: Non-linear. All levels unlocked at runtime.

- Pedagogical Target: Age-appropriate cognitive challenge (9 years old).

- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.

- Data Integrity: Strict mathematical logic. Realistic numerical bounds.

- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.

2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.

3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.

4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.

5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{DistrictHealth} = \text{DistrictHealth} - \text{DecayRate} + (\text{AILabeledFunds} * 0.5) + (\text{ManualOverrideFunds} * 1.5)$. Manual overrides cost double the financial transaction fee.
- Debugger: "🔧" pauses the health decay timer to inspect the active classification cue weights.

P5

Act as an expert frontend game developer and educational designer. Write a complete game called "Tax Anomaly Inspector" designed for 10-year-old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students the critical framework of Human-in-the-Loop (HITL) AI Systems and Anomaly Detection. Students learn that AI flagging algorithms find outliers but often generate False Positives, requiring a skilled human auditor to verify evidence, correct the AI's bias, and deploy targeted digital probes.

Strict Tablet & Interface Constraints:

1. The View: A tax auditing desk. The left panel shows corporate data folders flagged red by an AI fraud detector. The center shows corporate wire histories. The right panel contains audit actions ("Dismiss False Positive", "Deploy Compute Probe", "Freeze Assets").
2. The Interface Metrics: Displays an "AI Confidence Index (%)" and a "Public Trust Meter."
3. Interaction: Native touch. Tap folders to reveal transaction data, then select the correct audit action button based on numerical evidence.

Game Structure & Level Logic (Teaching HITL Auditing & Verification Probes):

- Level 1 [Verify the Flag]: Instruction: "The AI flagged a company for fraud with 90% confidence. Look at their wires: they sent money to an unknown offshore shell company! Tap 'Deploy Compute Probe' to trace the wire."
- Level 2 [The False Positive Trap]: The AI flags a bakery for spending money abnormally. Inspection reveals they just bought an expensive holiday oven. Avoid a public trust penalty by clicking 'Dismiss False Positive'.
- Level 3 [The Shell Game]: A shell corporation splits an illegal offshore transfer into 5 tiny, separate payments to dodge detection filters. Trace the split vectors across the wire panel and deploy compute probes to link them together.
- Level 4 [Bias Correction]: The automated system begins flags matching a specific benign keyword pattern over and over due to algorithmic bias. Override 4 false flags in a row while tracking down 1 true asset-hiding violator.

- Level 5 [The Master Audit Hunt]: Scan a fast-moving ledger of 8 flagged corporations. Successfully resolve all false alarms and freeze the assets of all verified fraudulent nodes before your operational clock expires.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (10 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify all game constants and math formulas use valid, realistic values.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{IsFraud} = \text{Sum}(\text{OffshoreWires}) > (\text{AllowedThreshold} * 2.5)$. Wrong human decisions decrease the 'PublicTrustMeter' by 20 points.
- Debugger: "🔑" reveals an encrypted data layer showing the absolute True/False status of all current nodes.

P6

Act as an expert frontend game developer and data science instructor. Write a complete game called "Macro Auditing Sandbox" designed for 11-12 year old students.

The game must be fully playable and functional as a standalone HTML5 application.

Explicit AI Pedagogical Goal:

This game explicitly teaches students how advanced machine learning algorithms perform Predictive Forecasting and Sandbox Modeling in macroeconomics. Students learn how an AI evaluates complex, multi-variable feedback loops to project the long-term impact of fiscal decisions on inflation, GDP growth, and market security.

Strict Tablet & Mathematical Rules:

1. The Map: A structural flowchart of a city's economic ecosystem divided into three live data wheels: Market Index, Infrastructure, and Treasury.
2. The AI Sandbox Panel: A live line graph showing a dotted "AI Forecast Path" projecting 10 seconds into the future. Changing policy sliders instantly updates the dotted prediction line before the real economy shifts.
3. Interactive Sliders: Three macroeconomic controllers: "Tax Rate Slider", "Infrastructure Investment Slider", and "AI Audit Intensity Slider."

Game Structure & Level Logic (Teaching Predictive System Dynamics):

- Level 1 [The Inflation Shock]: Instruction: "A sudden economic shock causes the AI Forecast Path to predict massive inflation! Raise the 'AI Audit Intensity Slider' to root out tax evasion and stabilize the projected trend line."
- Level 2 [The Infrastructure Dip]: The AI forecast shows public roads crumbling in 10 seconds due to low funds. Lower the Tax Rate slightly to stimulate business while sliding funding into Infrastructure to lift the forecast.
- Level 3 [The Deficit Loop]: High audit intensity is catching fraud, but running the computing models is draining the city treasury. Find the exact mathematical balance on your sliders where audit revenue exceeds compute overhead.
- Level 4 [The Hyper-Stagflation Crisis]: A massive systemic market contraction occurs. Use the real-time predictive sandbox line to test multiple extreme slider combinations until you discover the unique route that stabilizes both employment and GDP projections.
- Level 5 [Total Network Equivalence]: Run the complete municipal sandbox for 60 seconds through random economic shock events. Maintain an aggregate Economic Stability Rating above 85% by constant micro-adjustments based on changing AI forecasts.

SYSTEM CONSTRAINTS: GAME ARCHITECTURE

- Deployment: Standalone HTML5/CSS3/JavaScript.
- Target Hardware: Tablet devices.
- UI/UX: Native touch interactions, responsive layout, fluid scaling.
- Difficulty Scaling: Minimum 5 discrete levels. Incremental complexity.
- Level Access: Non-linear. All levels unlocked at runtime.
- Pedagogical Target: Age-appropriate cognitive challenge (11-12 years old).
- Domain Theme: Strong focus on Artificial Intelligence concepts, logic, and mechanics.
- Data Integrity: Strict mathematical logic. Realistic numerical bounds.
- Clear instructions for users to know how to play the game.

SYSTEM MANDATE: SELF-CORRECTION & AUTOMATED VALIDATION

Before final code output, execute the following execution block:

[START VALIDATION BLOCK]

1. STATIC ANALYSIS: Scan generated HTML/JS for syntax errors and broken tags.
2. LOGIC CHECK: Trace data flow to confirm win/loss conditions are reachable.
3. NUMERICAL AUDIT: Verify predictive forecasting matrices and dynamic differential equations.
4. SIMULATION RUN: Programmatically simulate user input for Level 1 and Level 5 to catch runtime crashes.
5. REFACTOR: Automatically correct any failed validation metrics before compiling final response.

[END VALIDATION BLOCK]

Mandatory Logic Validation:

- Math: $\text{PredictedStability} = (\text{TaxRate} * 0.2) + (\text{Infrastructure} * 0.4) + (\text{AuditIntensity} * 0.4) - (\text{SystemInflationFactor} * 1.8)$.
- Debugger: "🔧" freezes the macroeconomic clock to let users input direct numerical values into the algorithmic forecasting array.